

*An information system to support
interpretation of data from an Inertial
Measurement Unit sensor used in diagnostics
of Benign Positional Paroxysmal Vertigo*



AALBORG UNIVERSITY
DENMARK

GROUP HST-23-ST-5-5406

BIOMEDICAL ENGINEERING AND INFORMATICS 5. SEMESTER 2023

AALBORG UNIVERSITY



AALBORG UNIVERSITY
DENMARK

Department of Health Science and Technology

Biomedical Engineering and Informatics

Selma Lagerløfs Vej 249

9220 Aalborg Øst

<http://www.hst.aau.dk/>

Title:

An information system to support interpretation of data from an Inertial Measurement Unit sensor used in diagnostics of Benign Postional Paroxysmal Vertigo

Project:

Semesterproject 5. semester

Project period:

1 sep. 2023 - 20 dec. 2023

Projectgroup:

Hst-23-st-5-5406

Participants:

Alberte Kløve Skov Jensen

Anne Reinert

Kirstine Riisager Madsen

Natasja Toftgaard Garner

Signe Meiners Lau

Stine Wiggers

Supervisor:

Stephen Edward Rees

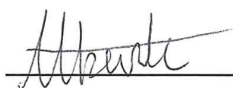
Abstract:

Benign Paroxysmal Positional Vertigo (BPPV) is a condition annually affecting 30,000 individuals in Denmark, characterized by the displacement of otoliths in the semicircular canals of the ear. BPPV presents itself as episodes of intense dizziness and vertigo. The reliability of the diagnosis of manual maneuvers depends on the execution. The specified angles, mean angular velocity, and duration must be adhered to provide optimal reliability in the diagnostic process. Addressing this problem, an information system for BPPV diagnosis has been developed in collaboration with an external partner from the *Balance & Dizziness Center at Aalborg University Hospital*. The external partner has provided the project with data recorded from an IMU sensor for the Dix-Hallpike and Supine Roll maneuvers. Through conversation with the external partner, requirements have been developed. To address these requirements object-oriented methods have been employed. The test results indicate that the majority of the requirements have been obtained. Suggestions for solutions and ideas for further development are subsequently deliberated.

Page numbers ex. Appendix: 64

Page numbers incl. Appendix: 77

Finished: 15.12.2023



Alberte Kløve Skov Jensen

aksj21@student.aau.dk



Anne Hærvig Reinert

areine20@student.aau.dk



Kirstine Riisager Madsen

krma21@student.aau.dk



Natasja Toftgaard Garner

ngarne21@student.aau.dk



Signe Meiners Lau

silau21@student.aau.dk



Stine Wiggers

swigg21@student.aau.dk

Preface

This project has been developed by group hst-23-st-5-5406, in the fifth semester of Biomedical Engineering and Informatics. This project was developed in collaboration with our supervisor Stephen Edward Rees and external partner Malene Hentze, doctor and Ph.D. student, by *Balance & Dizziness Center, Aalborg University Hospital*. The collaboration has involved recurring meetings and email correspondence. The meetings led to an understanding of the product's goals and the ability to adapt it on an ongoing basis. Ultimately, our partner assessed and tested the extent to which the product had achieved its goals.

The theme of the project is *Processing Patient Data and Information* from which the project proposal *An information system to support interpretation of data from an Inertial Measurement Unit sensor used in diagnostics of Benign Positional Paroxysmal Vertigo* has been selected and assigned. The project is allocated 15 ECTS point and was written during the period from September 1st to December 20th, 2023.

The purpose of the project is to develop an information system that can calculate and visualize the requested values for our partner's Ph.D. thesis. In the development of the project, the programs Netbeans, MATLAB, yED, and UMLet were used for programming and diagram development. Overleaf has been used as the platform for writing the rapport.

Reading guide:

The project report begins with an introduction that briefly outlines the focus of the report, leading to the formulation of an initiating question. The initiating question leads to the foundation for the problem analysis which is delimited in a problem delimitation leading to the formulation of the problem statement. Following the problem statement is a solution strategy part composing the methods used in the project. Subsequently, requirements, software development, test, discussion, perspective, and conclusion will follow.

Throughout the report, there are used abbreviations, the words are written out the first time they are used. The abbreviation is written in brackets next to the word for example: Benign paroxysmal positional vertigo (BPPV). Subsequently, only the abbreviation will be used in the report.

Reference method:

The reference method *Vancouver* is used. Reference after periods refers to the entire section. For example "text. [source]". A reference that appears before a period refers to a sentence. For example "text [source].".

Contents

1	Introduction	1
I	Problem analysis	2
2	Benign Paroxysmal Positional Vertigo	3
2.1	Symptoms	3
2.2	Anatomy of the balance organ	3
3	Diagnosing Benign Paroxysmal Positional Vertigo	6
3.1	Nystagmus	6
3.2	Dix-Hallpike maneuver	7
3.3	Supine Roll maneuver for horizontal canal	8
3.4	Angular position and velocity	9
3.5	Technology for diagnosis	9
3.6	The repositioning chair	11
3.7	Treatment of Benign Paroxysmal Positional Vertigo	12
4	Problem delimitation	13
4.1	Problem statement	13
II	Solution strategy	14
5	Solution strategy	15
5.1	V-Model	15
5.2	Object-oriented method	16
5.3	Object-oriented analysis	17
5.4	Object-oriented design	20
5.5	Object-oriented programming	21
5.6	The MoSCoW model	23
III	Requirements	24
6	User scenario	25

6.1	User requirements	25
7	Development process	26
7.1	Use case diagram	26
7.2	Use cases	26
7.3	Activity diagram	28
7.4	Prototype	30
8	System Requirements	32
8.1	Functional requirements	32
8.2	Non-functional requirements	33
IV	Software development	34
9	Software development	35
9.1	Object-oriented programming implementation	35
9.2	Design	38
9.3	Structure and class interactions	45
V	Test	46
10	Test of the program	47
10.1	Test of FR1	47
10.2	Test of Dix-Hallpike	48
10.3	Test of Supine Roll	49
10.4	Test of non-functional requirements	51
10.5	User experience	54
10.6	Results	56
10.7	Test summary	58
VI	Discussion and Conclusion	59
11	Discussion	60
11.1	Future perspective	63
12	Conclusion	64
	Bibliography	65

VIIAppendix	69
A Particle repositioning maneuvers	70
B Visualizing of quaternions	72
C Visualization of precise head angular position	73
D Prototypes	74

1. Introduction

Benign paroxysmal positional vertigo (BPPV) is a condition where the affected person can experience vertigo, which is triggered by changes in head positions in different gravity directions. Vertigo is a sensation where the person's awareness of the environment changes, and it feels like the environment is spinning. The phenomenon originates in the semicircular canals in the inner ear, where crystals called otoliths move into these canals and disturb the normal flow of the vestibular fluid.[1] Even though BPPV is benign, persons with BPPV are markedly limited in their daily activities because of the vertigo[2].

The annual incidence of BPPV in Denmark is approximately 30,000 people[3]. BPPV is the most common vestibular cause of vertigo in adults where 60% is affected. Prevalence increases with age, with the average age of onset being around 60 years, and approximately two out of three patients are women.[4][5]

Based on the above, the following initiating question will be investigated:

What are the characteristics of BPPV and which methods are used to diagnose BPPV?

Part I

Problem analysis

2. Benign Paroxysmal Positional Vertigo

In the following chapter, the characteristics of BPPV will be described with a focus on symptoms and anatomy.

2.1 Symptoms

The most significant symptom of BPPV is experiencing dizziness or vertigo when changing to new positions [6]. Dizziness includes various sensations, including feeling weak and faint, while vertigo is characterized by the sensation that the surroundings are spinning. [7] The symptoms normally last a few seconds to a few minutes[6]. The person can experience a period of hyperacute vertigo seizures, which last 10-30 seconds and may be accompanied by nausea or vomiting. The symptoms of vertigo may include fatigue, indicating that with repeated occurrences, they can be weakened or disappear completely. The symptoms can be retriggered within a period of up to half an hour if vertigo is provoked again.[4]

People with head trauma or infections in the inner ear are more likely to develop BPPV [2]. Clinical examinations also suggest that hormonal fluctuations, particularly during menopause, may contribute to the development of BPPV.[8]

2.2 Anatomy of the balance organ

To gain an understanding of why symptoms of BPPV occur, it's essential to have knowledge about the anatomy of the ear, where the balance organ is situated.

The inner ear consists of semicircular canals, otolith organs, and cochlea, as shown in figure 2.1. These components together are referred to as the membranous labyrinths whose purpose is to maintain balance and hearing.[9] The otolith organs are two membranous sacs called utricle and saccule where the utricle primarily senses horizontal motion and the saccule senses vertical motion.[10] As seen in the figure 2.1, the utricle is connected to the semicircular canals, while the saccule is connected to the cochlear duct.[9] Furthermore, the purpose of the otolith organs is to stabilize gaze and postural control.[6]

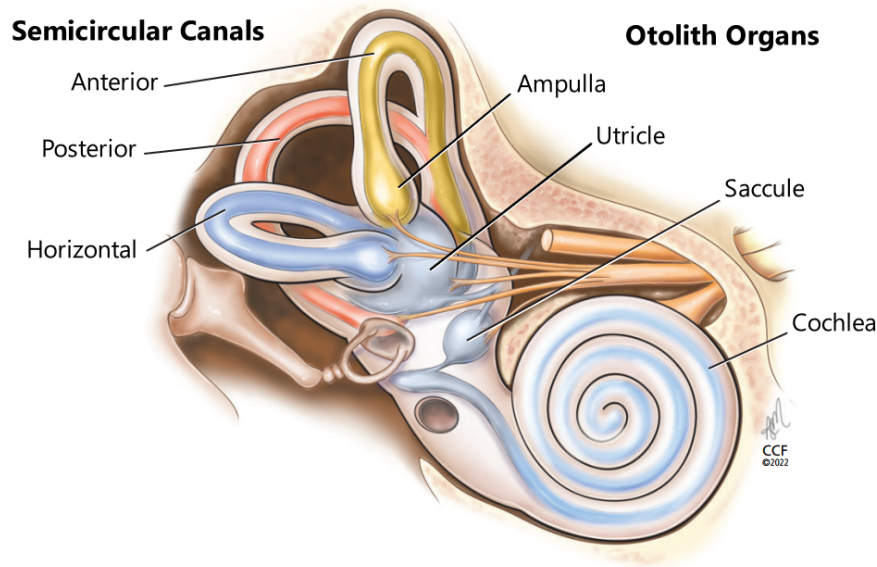


Figure 2.1: The structure of the inner ear. To the left are the three semicircular canals, in the center are the otolith organs; the utricle and saccule, and to the right is the hearing organ; the cochlea. The image has been modified from [6].

Within the semicircular canals, the semicircular ducts are situated. These ducts register the angular acceleration of head movements. There are three of these ducts: one in the anterior, one in the horizontal, and one in the posterior orientation, all situated superoposterior to the vestibule.[9]

In these ducts is a fluid called endolymph that moves with changing speed and acceleration as the head moves. Special sensory receptors located in structures called ampullae detect these movements. Ampullae are slightly thicker than the canals and serve as the endings of each canal. These sensory signals are then transmitted to the brain for processing through the vestibular nerve. This process allows for a sense of the body's orientation and movement in space, ultimately contributing to balance.[9]

The vestibulo-ocular reflex serves the purpose of gathering information about the head's position together with the sense of balance. The reflex works in conjunction with the eyes to stabilize images on the retina, when the head is rotating or turning from side to side. Therefore, it's possible to perceive stationary objects even when the head and body are in motion.[11]

2.2.1 Otoliths

The utricle and saccule are often referred to as the otolith organ due to the presence of small structures called otoliths. Figure 2.2 illustrates multiple otoliths in the otolith organ and a single otolith shown to the right.[10]

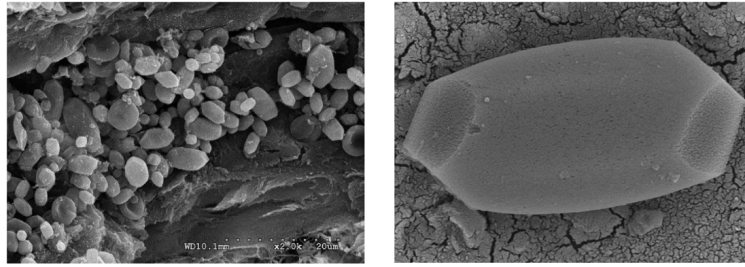


Figure 2.2: To the left, multiple otoliths can be observed within a structure. To the right, is a close-up view of a single otolith.[10]

Otoliths are composed of calcium carbonate crystals, and their gravitational force influences the membranes within the utricle and saccule. This force stimulates the hair cells whose purpose is to register signals about motion and head position. The signals then travel via the vestibular nerve to the cerebellum where they're processed. If one or more otoliths have come loose they can float into the semicircular ducts which will disturb the normal function of the semicircular ducts, resulting in BPPV.[10]

BPPV is categorized into two subgroups: canalolithiasis, also known as posterior-canal-type BPPV, and cupulolithiasis, also referred to as horizontal-canal-type BPPV. In canalolithiasis, the otolith detaches from the otolithic membrane and is displaced into the endolymph of the canal. Cupulolithiasis occurs when the otolith debris floats in the semicircular canal and rests on the cupula. The cupula is located in the ampulla, as depicted in the figure 2.1.[1]

3. Diagnosing Benign Paroxysmal Positional Vertigo

To diagnose BPPV, different maneuvers can be performed, both manually or supported by technology. This chapter will describe these different maneuvers and their advantages and disadvantages including technology that supports the diagnosing maneuvers.

3.1 Nystagmus

During the diagnostic process, an eye movement pattern known as nystagmus is observed to confirm the diagnosis of BPPV[6]. Nystagmus is repeated rhythmic eye oscillation. Stimulation of the semicircular ducts most commonly causes jerk nystagmus, consisting of a slow phase in one direction and a rapid phase back to the original position. The nystagmus is named after the direction of the fast phase. It can be horizontal, vertical, rotatory, or any combination of these. Furthermore, the nystagmus can be geotropic or apogeotropic. Geotropic is when nystagmus fast phase beats toward the affected ear and apogeotropic is when nystagmus beats away from the affected ear.[12]

Table 3.1 illustrates the different nystagmus variations. Furthermore, it's seen in the table that the direction of the nystagmus is affected by which canal the otoliths are placed.[6] Dix-Hallpike and Supine Roll are both provocative maneuvers that will be described in the next section.

Provocative maneuver	Nystagmus direction	Affected canal
Dix-Hallpike	To the affected side and up	Posterior
	To the affected side and down	Anterior
Supine roll, right ear down	Right (geotropic)	Horizontal. (The head-turn direction eliciting stronger nystagmus indicates the affected horizontal canal)
Supine roll, left ear down	Left (geotropic)	
Supine roll, right ear down	Right (apogeotropic)	Horizontal. (The head-turn direction eliciting weaker nystagmus indicates the affected horizontal canal)
Supine roll, left ear down	Left (apogeotropic)	

Table 3.1: Observed nystagmus patterns based on semicircular canal involvement. The table has been modified from [6].

3.2 Dix-Hallpike maneuver

The Dix-Hallpike maneuver contains two positional changes. The purpose is to observe nystagmus pattern to determine which canal is affected. The Dix-Hallpike maneuver is performed in the following steps:

1. The patient's head is turned 45°.
2. The patient is moved from sitting to supine position at -120°.
3. The eye movements are observed to see if there are any signs of nystagmus.
4. The patient is moved from supine to sitting and the maneuver will be repeated with the patient's head turned 45° to the other side to examine both the right and left ear. [6]

Dix-Hallpike maneuver for posterior canal

The posterior canal is the most frequently affected canal due to its anatomical placement and because the otoliths have easier access to the posterior canal. The nystagmus is shown by torsional ocular and will last less than 60 seconds and have a latency between 1 and 40 seconds. [13] Furthermore, the rotational movements will be clockwise for the left ear affected and counterclockwise for the right ear.[14] This nystagmus pattern should be observed when the patient moves from sitting to supine position [6]. In figure 3.1 the Dix-Hallpike maneuver is shown with the eyes moving as the otolith is floating in the posterior canal.

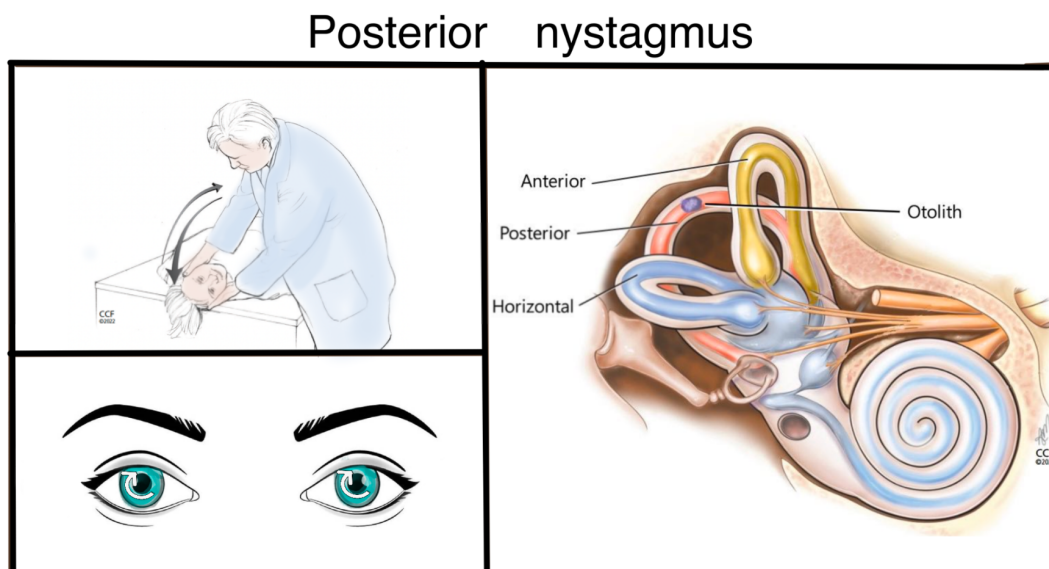


Figure 3.1: The Dix-Hallpike maneuver is used to detect otoliths in the posterior canal. Posterior nystagmus is shown in rotational movements of the eyes and is observed by the healthcare professional. The image has been modified from [6]

Dix-Hallpike maneuvers for anterior canal

The presence of an otolith in the anterior canal can lead to another pattern of nystagmus, which is shown by a vertical down-beating motion. In figure 3.2 the healthcare professional is moving the

patient's head where nystagmus in a vertical direction is observed. The otolith is seen in the anterior canal. Normally, the anterior nystagmus is more explicit and less intense in its movements than the posterior nystagmus.[14]

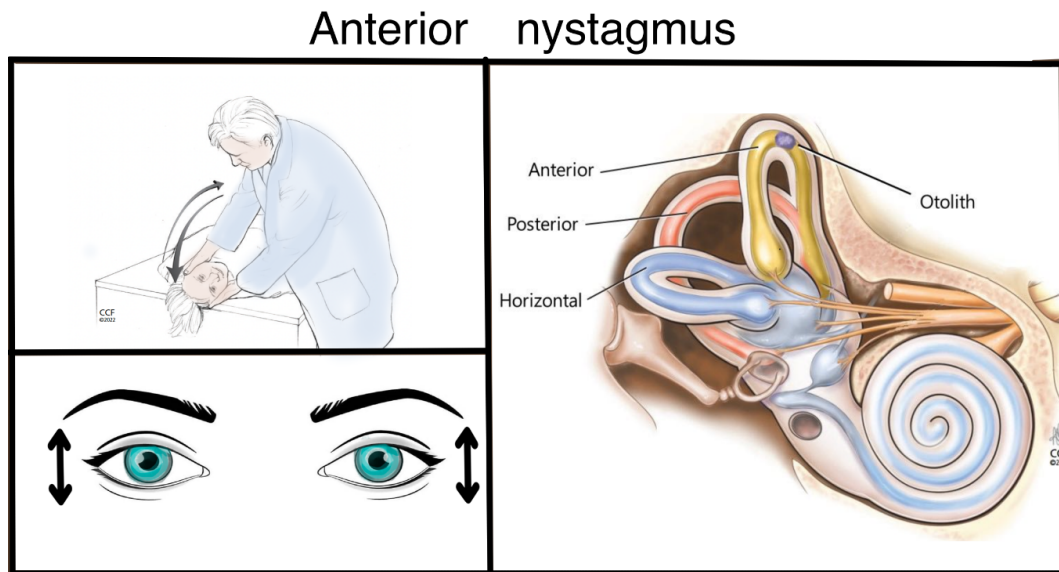


Figure 3.2: The Dix-Hallpike maneuver is used to detect otoliths in the anterior canal. Anterior nystagmus is shown in vertical movements of the eyes and is observed by healthcare professionals. The image has been modified from [6].

3.3 Supine Roll maneuver for horizontal canal

The Supine Roll maneuver is recommended as the second screening maneuver after the Dix-Hallpike maneuver because it's more common to have otoliths in the posterior and anterior canals than in the horizontal canal [6]. The Supine Roll maneuver is used to diagnose horizontal canal BPPV by eliciting horizontal nystagmus and is performed in different phases, see figure 3.3 [15]. The Supine Roll maneuver is performed in the following steps:

1. The patient is in a supine position where the head is inclined forward at an angle of 60° .
2. The patient's head gets turned 90° , quickly to the right, where the patient is held.
3. The eye movements are observed to see if there are any signs of nystagmus.
4. After 30 to 60 seconds the patient's head is quickly turned to the left.
5. The eye movements are again observed to see if there are any signs of nystagmus.
6. The patient's head is turned back to the starting point where the patient is held in a sitting position. [6]

If horizontal nystagmus is observed the patients can be diagnosed with BPPV in the horizontal semi-circular canals which is illustrated in figure 3.3.[6]

Horizontal nystagmus

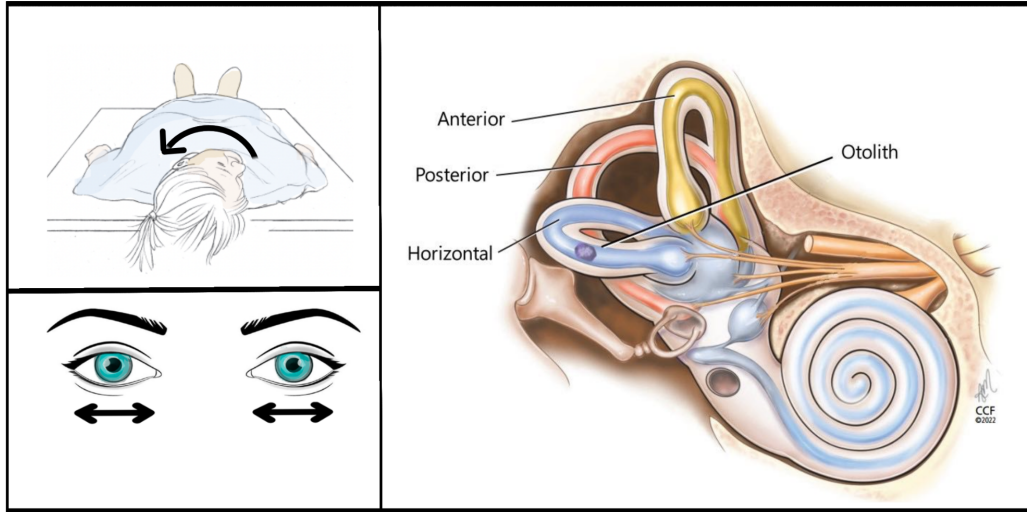


Figure 3.3: The Supine Roll maneuver is used to detect otoliths in the horizontal canal. Horizontal nystagmus is shown in horizontal movements of the eyes and is observed by the healthcare professional. The image has been modified from [6].

3.4 Angular position and velocity

During the execution of the maneuvers for diagnosis of BPPV, there is an accurate description in steps of how the maneuvers are performed. In the steps, certain angles must be hit to perform the diagnosis of BPPV. Achieving these angles in the manual test can be challenging due to the difficulty of precisely assessing when the angles are reached. In addition to achieving precise angles, there is also a specific angular velocity and timeframe between the shifts that must be observed. These timeframes require the performer to be accurate during the performance. It takes practice to hit the specific angles within the right timeframe. It can affect the result of the diagnosis if this accuracy isn't achieved during the execution.[16]

3.5 Technology for diagnosis

To measure the head position and movement in real-time an inertial measurement unit (IMU) sensor can be applied. The IMU sensor is a small device, that typically consists of a gyroscope, accelerometer, and sometimes a magnetometer where the combination of linear accelerations and angular velocities give 3D positions. The linear accelerations are along each axis and angular velocities are around each axis.[17, 18] The 3D position is measured in quaternions which represent rotation in 3D. Quaternions are represented in two components which are the vector component (x, y, z) and scalar component (w). The vector component describes the three dimensions and the scalar component describes the rotation around the dimensions.[19] The rotation around the vectors (x, y, z) is called pitch, roll, and yaw, which is illustrated in figure 3.4 [20].

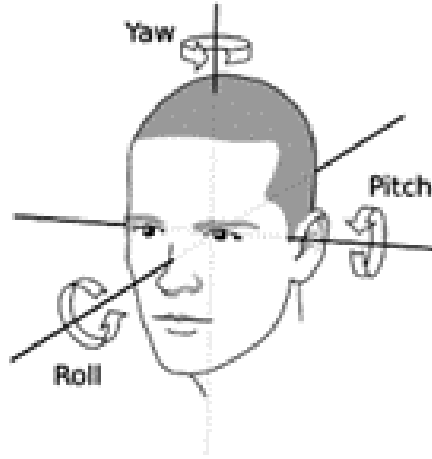


Figure 3.4: The three dimensions of movement; pitch, roll, and yaw. [21]

The precise angles corresponding to the axes and sides for Dix-Hallpike and Supine Roll maneuver are listed in figure 3.5. Furthermore, the graphs that illustrate the precise head angular positions are in appendix C.

	Pitch	Roll	Yaw
Supine Roll Test (test af de horisontale buegange)			
Supine position	-60	0	0
Right side	-60	0	+90
Left side	-60	0	-90
Dix-Hallpike right (test af de bagre og forreste buegange)			
Upright	0	0	+45
Supine position	-120	0	+45
Dix-Hallpike left			
Upright	0	0	-45
Supine position	-120	0	-45

Figure 3.5: The different positions for each dimension pitch, roll, and yaw during the maneuver[22].

The IMU sensor can be implemented in different systems or technologies such as infrared goggles. With the IMU sensor implemented on the goggles, it's possible to analyze the data, recorded from the IMU sensor to check if the precise angles are hit. This is necessary, during the execution of the Dix-Hallpike and Supine Roll maneuvers, as described in section 3.2 and 3.3.

As mentioned the IMU sensor can be implemented on infrared goggles. The purpose of the goggles is to record and replay eye movements. Using infrared goggles, it's possible to measure nystagmus.[23] Figure 3.6 visualizes the implementation of the IMU sensor on the goggles.



Figure 3.6: The IMU sensor implemented on the infrared goggles. The image has been modified from [23].

With the infrared goggles, the eyes are in darkness which removes the visual fixation. Under the different maneuvers, it's possible to see the eye movement through video on a computer screen. With the video, it's possible to compare the recordings over time and monitor changes.[24]

3.6 The repositioning chair

Some patients are difficult to diagnose, with the manual maneuvers performed on the bed. The reasons could include a lack of mobility or the operator's inability to handle the weight of a patient. About 10% to 20% of patients with BPPV can't be diagnosed and treated with manual maneuvers.[25]

These factors make the maneuvers challenging to perform, and as a result, a mechanical repositioning chair has been designed to address these challenges. The chair makes it possible to move patients to any required position manually while their eyes are being monitored for nystagmus. One of the most used repositioning chairs is the Vertigo Treatment and Rehabilitation (TRV) chair developed by Interacoustics. Figure 3.7 shows the TRV chair and how the patient with the infrared goggles is fastened.[25]



Figure 3.7: The TRV repositioning chair developed by Interacoustics, and how the patient is fastened in the chair with the infrared goggles on [26].

The TRV chair also makes it possible to rotate the patient 360° manually, which allows the healthcare professional to lock the patients in any position which makes it possible to observe each semicircular

canal. The TRV chair enables exact and reproducible positioning of the patient independent of the examiner.[27]

3.7 Treatment of Benign Paroxysmal Positional Vertigo

A comprehensive diagnosis will enable the administration of the most suitable treatment for the patients. After the patient is diagnosed with BPPV, the purpose of the treatment is to remove the otolith from the semicircular canal and back to the utricle. Depending on which of the three canals the otolith is located, different treatments will be applied.[6] The preferred treatment method is the particle repositioning maneuver (PRM). Repositioning helps move the otolith back to the utricle with different head position changes. PRM consists of different therapy maneuvers such as Semont, Epley, and log roll. [28] The TRV chair is also used for treatment and has some advantages as mentioned earlier in section 3.6. For further information about the performance of the three maneuvers see appendix A.

4. Problem delimitation

Based on the problem analysis, the reliability of the manual maneuvers depends on the execution in connection with the diagnosis of BPPV.

The healthcare professional must hit precise angles with the predetermined time duration for the position shifts during execution. Furthermore, the angular velocity during the manual maneuvers plays a significant role in the diagnosis process. These factors affect the performance of the manual maneuvers and thereby the reliability of the diagnosis if the factors are not performed correctly, which is described in figure 3.5 and in section 3.2 and 3.3. This makes it difficult for the healthcare professional to perform the diagnosis and give the patient the correct diagnosis.

As emphasized by the external partner from the *Balance & Dizziness Center, Aalborg University Hospital*: "When performing the tests manually, the reliability of these diagnostic tests most likely depends on how precise and fast the head positions are executed. This might explain why different clinics and doctors come to different conclusions."

Currently, there are no information systems designed to analyze whether the maneuvers are done precisely. Therefore a research team from the *Balance & Dizziness Center, Aalborg University Hospital* has requested an information system that can display angles, time duration for shifts, and angular velocity. The research team used an IMU sensor and extracted data during the maneuvers which is given in quaternions.

Based on the problem delimitation, the problem statement is derived. The problem statement focuses on the development of an information system, whose purpose is to enable the use to evaluate the precision of manual maneuvers.

4.1 Problem statement

How can a digital information system be developed to calculate values which allow the user to analyze whether the maneuvers for the diagnosis of BPPV, are performed precisely?

Part II

Solution strategy

5. Solution strategy

Based on the problem statement an information system will be developed. The collaboration with the external partner leads to an agile working approach with an emphasis on iterative system development. To develop the information system, the following methods are used.

5.1 V-Model

The V-Model, also known as the verification and validation model, comprises two phases. The V-Model is illustrated figure 5.1. In the verification phase, it's important to understand the customers' expectations and the user requirements for the product. The validation phase is used in projects to work with the process of evaluating the product, where it's found whether the specified requirements are met. In the validation phase, testing is done for functional-, and non-functional requirements by testing the product. Finally, it's decided whether the software meets the customer's requirements and expectations. [29]

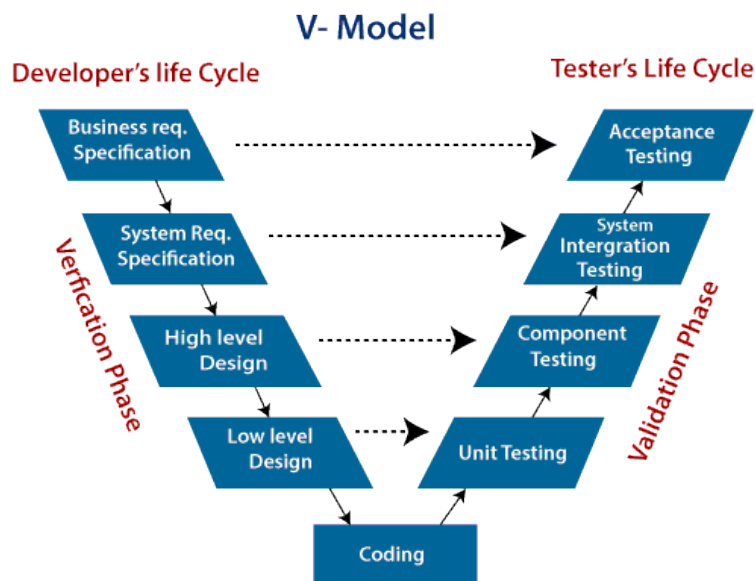


Figure 5.1: The V-Model contains the verification and validation phase.[29]

Below are listed the elements of the verification phase:

- *Business Requirement Specification:* Contains detailed communication to understand the customer's expectations and exact requirements for the product from the customer's perspective. This is an important activity, as most of the customers are not sure of their exact needs.

- *System Requirement Specification:* In this stage the user requirements are studied, by analyzing and interpreting.
- *High level Design:* Contains modules like interface relationships, dependencies, database tables, and technology. In this stage data transfer and communication between the internal modules and with the outside world and other systems is clearly defined.
- *Low level Design:* Contains detailed internal design for all the system modules. The system is broken down into small modules and the module's design is specified. The unit tests are an essential part of any development process and help eliminate errors at an early stage.
- *Coding:* Based on the requirements of the best suitable programming language. The guidelines and standards for coding must be observed. To check the performance the code goes through numerous code reviews. [29, 30]

Below are listed the elements of the validation phase:

- *Unit Testing:* Executed to eliminate errors at the code level or unit level. Unit testing is the testing at the code level and verifies that the smallest entity can function correctly when isolated from the rest of the codes or units. It can help eliminate some bugs at an early stage.
- *Component Testing:* Associated with the architectural design phase. These tests verify that the tested independently can coexist but also communicate together.
- *System Integration:* Associated with the system design phase. Testing ensures that the expectations of an application developer are met, like design, and software. The tests check the entire system's functionality and the communication of the system.
- *Acceptance Testing:* Associated with the business requirement analysis phase. Includes testing of the software product in the user atmosphere. The acceptance testing tests the problems with the various systems, like load and performance defects within the real user atmosphere. [29, 30]

5.2 Object-oriented method

Object-oriented method (OOM) can be used during the development of a software product. The purpose of OOM is to split the system by using objects, classes, encapsulations, and messages. This makes it much more maintainable and easier to navigate for the programmer. The OOM makes it easier for the developer to customize the system to the user requirements and for future user requirements. [31]

There are three steps to OOM, object-oriented analysis(OOA), object-oriented design(OOD), and object-oriented programming(OOP) [31]. Through these steps, the Unified Modeling Language (UML) can be used to provide a visual illustration of an aspect of a system. The illustrations consist of many different types of diagrams that visually describe behavior, structure, functionality, or interactions.

The diagrams become more elaborate, the further the design process proceeds. These diagrams, as presented in figure 5.2, describe structural diagrams and behavioral diagrams used in the project. Structural diagrams include class diagrams, and behavioral diagrams include activity diagrams, sequence diagrams, and use case diagrams.[32]

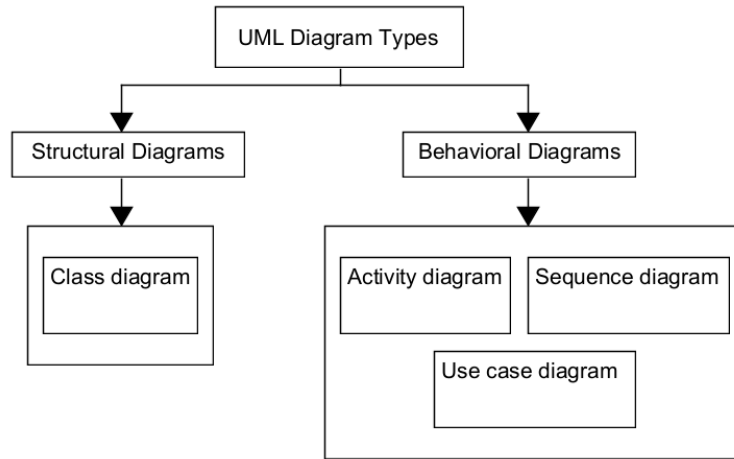


Figure 5.2: Overview of the used UML diagrams in the project.

5.3 Object-oriented analysis

The analysis process is based on the users's vision of a potential solution. Different methods are used to analyze the user requirements. The methods that are used in the OOA process are the use case diagram, use cases, activity diagram, and class diagram.[31]

5.3.1 Use case diagram

A use case diagram is a behavior diagram that illustrates the user scenario. It describes the system's behavior, functional requirements, and is developed from the user's perspective. The use case diagram provides a graphical overview of various use cases, which represent the methods and functions that a system should incorporate.[32]

To describe the process of creating and designing a use case, the following figure 5.3 is provided. Initially, one or more actors with a connection to a system or any software application are identified and illustrated by stick figures. The gray rectangle represents the system, while actors should be positioned outside this box. The white ovals illustrate the use cases, which are the functions that the system should provide for user interaction. Furthermore, a use case can be marked as an 'extends,' indicating that it's an optional addition to the primary use case. If a requirement is marked as an 'includes,' it must be incorporated into the primary use case.[32]

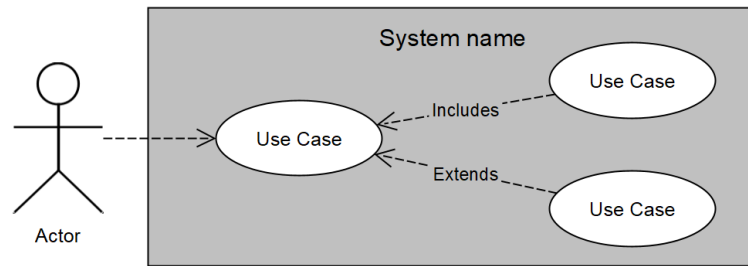


Figure 5.3: A use case template that includes actors, use cases, and their relationships.

5.3.2 Use cases

Following the completion of the use case diagram, use case tables are created. These structured tables provide descriptions of each use case. The primary purpose is to describe how the system responds to various user scenarios and interactions. Use case tables serve as a central tool for systematically organizing, documenting, and communicating the multifaceted requirements.[32]

Figure 5.4 illustrates the procedure for developing a use case as a table. On the left, various elements such as use case name, purpose, and main course are displayed. These elements are filled in the blank spaces on the right.

Use case name	
Use case ID	
Purpose	
Primary actor	
Prerequisite	
Main course	
Alternative course	
Succes criteria	
Minimum criteria	

Figure 5.4: A template for a use case table.

5.3.3 Class diagram

The class diagram is a structural diagram designed to model the static structure of a system, as depicted in figure 5.2. Class diagrams serve the purpose of organizing and defining classes, including their associated methods, and helping in the understanding of problem domain requirements.

Classes and their associated methods can be identified based on the nouns and verbs found in the use cases. Figure 5.5 visualize how classes are represented within boxes, with their corresponding attributes, operations, and methods listed. A class represents an object that shares the same structure and behavior. Classes can be connected with arrows, and some classes may have subclasses. Class diagrams are also utilized to illustrate the behavioral patterns within the system and their hierarchy.[32]

Class name
Attributes
Methods

Figure 5.5: A template for a class diagram that describes the structure of how the class name, attributes, and methods should be formatted.

During software development, code can be generated based on class diagrams, and conversely, class diagrams can be derived from code [32]. Designing and reusing classes reduces the likelihood of errors and simplifies error detection. Together it will state how the classes will interact and what purpose they have. [31] The ideal class diagram is both precise in terms of methods and flexible, allowing for the easy and comprehensible addition of more and new classes in the future [32].

5.3.4 Activity diagram

An activity diagram is typically used in OOA and shows a visualization of the process flow and the sequence of actions. In activity diagrams, specific symbols are employed to signify various functions or elements. Figure 5.6 illustrates a template for an activity diagram. Some of the more common symbols are swimlanes, a filled circle, a circle with a cross, and a diamond symbol. Swimlanes are used to create an overview of which actors are active in the various parts of the activity diagram. A filled circle represents a start declaration and a circle with a cross or a filled dot inside it represents the end declaration. Arrows indicate the direction of the workflow. The advantages of using an activity diagram are that it can show concurrent or parallel flows as well as alternating flows. There are different phases where an activity diagram can be helpful, before starting a project, during the requirements phase, and the analysis and design phases. [32]

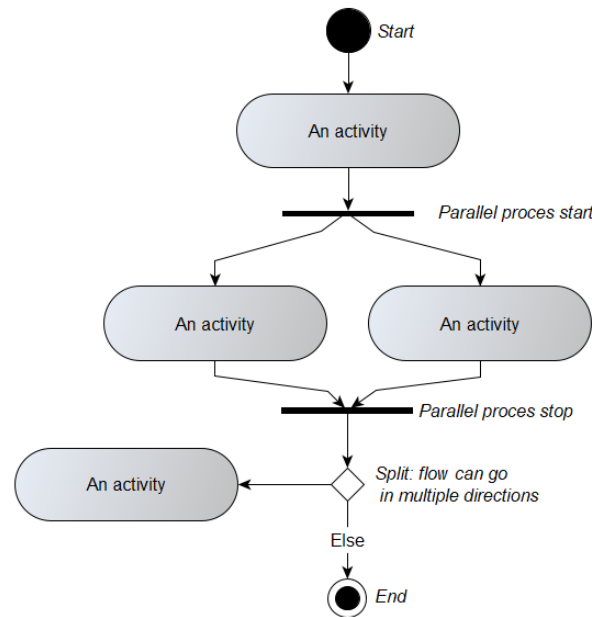


Figure 5.6: Depiction of how an activity diagram can be constructed.

5.4 Object-oriented design

OOD is the process between domain models which is made during the OOA, and the code written in OOP. In this process, the models become more detailed and more code-oriented. The class diagrams drafted in the OOA will be elaborated in more detail and more code-oriented classes can be implemented in the program. To help get a better understanding of the classes, sequence diagrams provide an interpretation of the link between the class diagrams and how these will become classes in code. At the end of the design process, it will typically be possible to form a general view of the final product which leads to an elaboration of a prototype.[31]

5.4.1 Sequence diagram

The sequence diagram is used in OOD and OOP with a focus on behavior. A sequence diagram illustrates the sequence of messages between objects in an interaction and shows the control structures between objects. Sequence diagrams are created by diagram elements, as shown in figure 5.7. These elements can be lifelines or messages, which are observed behavior. Lifelines are stippled lines and represent the objects that participate in an interaction. A message defines a specific kind of communication between instances in an interaction. The instances are represented by a rectangle, which contains the conditional structures that affect the flow of messages. Classes are defined with an underline in the instance. The instance is active where the grey box is marked on the lifeline. The message between the active instances is illustrated with arrows. To return the message an arrow goes the opposite way. The cross at the end of the lifeline defines that the instance lifeline has ended. [32]

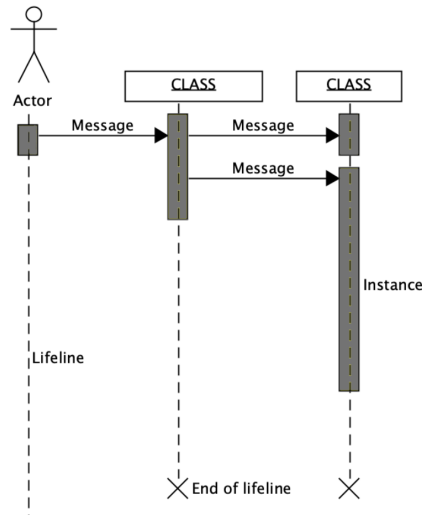


Figure 5.7: The different elements of a sequence diagram.

5.4.2 Prototype

A prototype serves as a tool in the process of product development and analysis. There are different elements in developing a prototype which include design, sketches, electronics, and software programming. Prototype produces a product that is highly adaptable and can be easily modified. Prototypes include varied aspects of an intended design and can be divided into low-fidelity and high-fidelity. [33]

A low-fidelity prototype is a basic representation of the initial product concept. It's created in the early stages of the design process to validate the product's concept through conversations and discussions with users. Low-fidelity prototypes can be in the form of sketches on paper or digital representations. The advantages of low-fidelity prototypes include their quick production, making it easier to modify and incorporate new features as per user requirements. [34]

A high-fidelity prototype is an interactive and refined representation of a product. This prototype undergoes user testing to assess its intuitiveness, and if it receives approval from the customer, it advances to the development stage. The benefits of using high-fidelity prototypes include the ability to thoroughly test both functionality and components such as input fields, graphic elements, and filters. Furthermore, it allows for the validation of all current hypotheses.[34]

5.5 Object-oriented programming

OOP is built upon three basic ideas: objects, messages, and classes. Classes are transformed into code that can be incorporated, evaluated, tested, and reused. [31] In OOP classes are user-defined data types, that define the state and behavior of an object. Classes can be grouped into classes and subclass hierarchies with a special characteristic by adding to the inherited behavior. Objects are

instances of a class that are defined with specific data. Objects communicate by sending messages, and objects respond by carrying out their methods. Methods are functions that are defined inside a class that describe the behaviors of an object. Attributes represent the state of an object and are defined in the class template. Objects need attributes to describe them. [35, 31]

5.5.1 Design patterns

Design patterns are developed by software engineers over time and are obtained through numerous trials and errors. The purpose of design patterns is to make flexible and reusable software and to make software that programmers can understand and is easy to communicate with by structuring classes and objects in specific ways. [36]

In the field of software engineering, various design patterns are available for use. There are creational, structural, and behavioral design patterns:

- *Creational*: Offer a way to create objects by abstracting the creation logic, avoiding direct instantiation using the new operator. This provides flexibility in determining which objects to create for a given use case.
- *Structural*: Focus on class and object composition, utilizing the concept of inheritance to compose interfaces and define methods for combining objects to achieve new functionalities
- *Behavioral*: Address communication between objects. [36]

Model-View-Controller

Model-View-Controller (MVC) organizes an application's logic into distinct layers, which each carry out a specific set of tasks. The layers interact with each other to ensure that the application's functionality is delivered in a coordinated and consistent manner. MVC is widely utilized in contemporary object-oriented programming, particularly for designing web applications. [37]

The MVC methodology separates an application's logic into three layers:

- *Model*: Contains the data and methods used to handle it, providing access to the data. Its responsibilities encompass the storage and retrieval of data, and it may also handle tasks such as data validation and other related operations.
- *View*: Captures events and user interactions with the graphical user interface (GUI) presentation. It incorporates various components for displaying data, such as buttons, tables, and text boxes that facilitate user interaction.
- *Controller*: Coordinates interactions between the View and Model, determining how the user interface responds to user input. It serves as the hub where all actions should be coordinated

and synchronized to ensure seamless operation. [37]

Developers can concurrently construct components without interfering with each other's progress, facilitating the reuse of these components. Additionally, it gives flexibility to deploy and maintain components independently.[37]

Singleton design pattern

A commonly used design pattern is the singleton, which is a creational design pattern. The Singleton pattern is a design pattern ensuring that a class has only one instance with global access to this singular occurrence. It revolves around a single class responsible for creating an object, ensuring a sole instance. The class provides a method for direct access to this object, eliminating the need for class instantiation. [38]

As an example, a singleton can represent scenarios where there are multiple windows, but only one Window Manager, or a collection of variables, such as a specific font type or the current user, where these variables typically are initialized at the beginning when a user logs in. [38]

5.6 The MoSCoW model

To prioritize the requirements of a project or product, the MoSCoW model is used. With this model, the requirements are prioritized based on *must have*, *should have*, *could have* and *won't have*. In the prioritization, the necessary requirements for the project are in *must have*, whereas the requirements that are not prioritized in the time frame for the project are in *won't have*. From *must have* to *won't have* the requirements become less important for the project. The table in figure 5.8 illustrates how requirements should be listed in MoSCoW prioritized as well as the meaning of each term.[39]

Abbreviation	Term	Meaning
Mo	Must have	Includes all the necessary requirements for successfully completing the project.
S	Should have	Requirements that are important to project completion, but not necessary.
Co	Could have	Desirable requirements but unimportant. Small impact on the project if left out.
W	Will not have	Requirements that are not prioritized and important in the project's time frame.

Figure 5.8: The MoSCoW model whose purpose is to prioritize the requirements.

Part III

Requirements

6. User scenario

The first step in the V-Model, as described in 5.1, in the verification phases is the Business Requirement Specification. An understanding of the user's exact requirements is expressed as a user scenario. The user scenario is based on the initial meeting with our external partner from the *Balance & Dizziness Center, Aalborg University Hospital*.

The user wants an information system that can calculate and display the mean angular position, the duration in time of the position shifts during the maneuvers, and the mean angular velocity. The user should subsequently be able to analyze their data using the system based on the calculated *values**.¹ According to the angular position, the user wants a graphic visualization as a function of time.

6.1 User requirements

User requirements have been developed based on the user scenario. The user requirements are presented in a list format, with the MoSCoW model used to categorize and prioritize them, as seen in section 5.6. The user requirements are illustrated in table 6.1.

ID	User requirements	MoSCoW
UR1	The user wants a system that can transform the measured quaternions from the IMU sensor to the angular position.	Mo
UR2	The user wants the mean angular position for each position in each maneuver visualized.	Mo
UR3	The user wants the duration of the position shifts during the maneuvers visualized.	Mo
UR4	The user wants the mean angular velocity during the position shifts visualized.	S
UR5	The user wants to be able to evaluate the <i>values*</i> from UR2, UR3 and UR4.	S
UR6	The user wants an interface that graphically visualizes the head positions as a function of time.	S
UR7	The user wants to export the <i>values*</i> .	Co

Table 6.1: User requirements listed in MoSCoW-Model prioritization.

¹*values** The mean angular position, the mean angular velocity, and the duration in time of the position shifts during the maneuvers.

7. Development process

This chapter covers the design development and process as derived from the methods described in the chapter 5. The use case diagram is presented with belonging use cases from which the activity- and class diagrams are developed. The fourth prototype is presented as a step to the final product.

7.1 Use case diagram

The use case diagram is illustrated in figure 7.1. The use case diagram is titled *Digital information system* and contains the researcher and several use cases. On the left, the researcher is depicted as the actor, with a line connecting to the main use case. From this main use case, two additional use cases are branched out in the form of *extends* relationships, represented by dashed arrows.

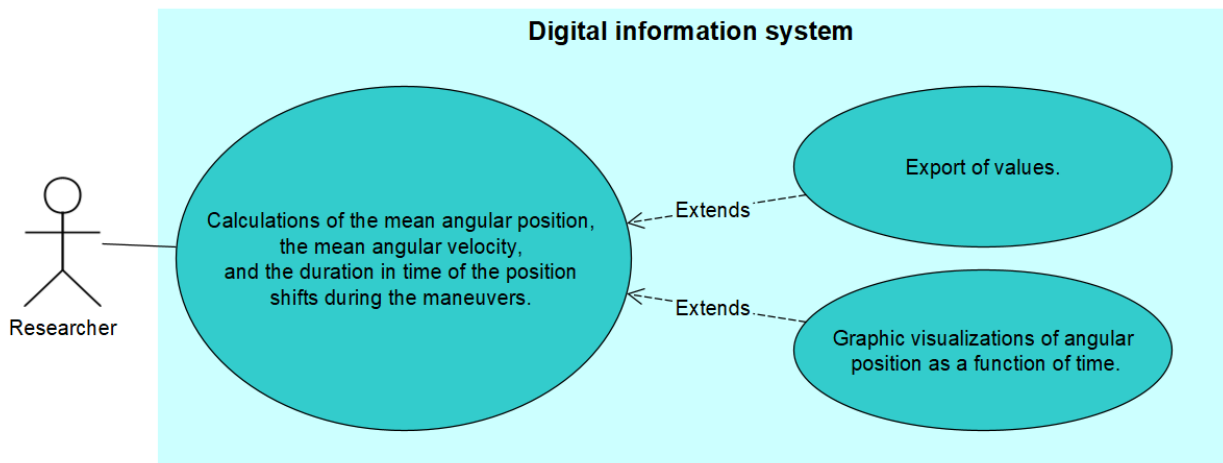


Figure 7.1: The use case diagram derived from the user scenario.

7.2 Use cases

The use cases are devised from the use case diagram. In the use case diagram, there are three use cases which will be described individually in each table in the following section.

Table 7.2 represents the first use case, also referred to as the main use case. In this use case, the user wants to have the data calculated and displayed.

Use case name	Calculate the mean angular position, mean angular velocity, and the duration in time of the position shifts during the maneuvers.
Use case ID	UC1
Purpose	The mean angular position, mean angular velocity, and the duration in time of the position shifts during the maneuvers is displayed so the user manually can evaluate these <i>values*</i> from the patient(s).
Primary actor	Researcher
Prerequisite	The user has datasets from the IMU-sensor in a csv file.
Main course	1. The user wants to choose which dataset(s) for which patient and maneuver(s) they want to have calculated. 2. The user wants the system to display the mean angular position, mean angular velocity, and the duration in time of the position shifts during the phases in the maneuvers. 3. The user now has the possibility to evaluate whether the maneuver(s) is performed precisely by manually analyzing the displayed <i>values*</i>. Extends: Export of <i>values*</i> (UC2). Extends: Visualize the graph of angular position as a function of time (UC3).
Alternative course	The user wants to know if an error occurs when choosing dataset(s) and maneuver(s).
Succes criteria	The user can see the calculated <i>values*</i> from the chosen dataset(s) and maneuver(s).
Minimum criteria	The <i>values*</i> will not be displayed if there is an error in the calculations.

Table 7.1: Use case 1 evolved from the use case diagram, as seen in figure 7.1.

Table 7.2 represents the second use case where exporting selected *values** to a CSV file will be an option for the user.

Use case name	Export of <i>values*</i> .
Use case ID	UC2
Purpose	To export <i>values*</i> for saving the data.
Primary actor	Researcher
Prerequisite	The user has a program downloaded on the computer which can open CSV files.
Main course	1. The user can export the data to a CSV file. 2. Afterwards the user can save the data on the computer for further use. 3. The user also wants the possibility to export specific, multiple, or combined datasets related to chosen data.
Alternative course	The user doesn't export the data.
Succes criteria	The data is exported to a CSV file.
Minimum criteria	If the data isn't exported correctly an error message will appear to make the user alert.

Table 7.2: Use case 2 evolved from the use case diagram, as seen in figure 7.1.

Table 7.2 represents the third use case where the user wants a graphic visualization of selected data.

Use case name	Visualize the graphs of angular position as a function of time.
Use case ID	UC3
Purpose	A graphic visualization of the angular position as a function of time is displayed so the user can analyze it manually according to what the user wants, along with the <i>values*</i> from UC1.
Primary actor	Researcher
Prerequisite	UC1 is occurred successfully.
Main course	1. The user wants to choose which dataset(s) for which maneuver(s) there have to be visualized. 2. The user sees the graphs visualized.
Alternative course	The user doesn't want to see the graphs illustrated.
Succes criteria	The user can see the choosen graph(s) visualized.
Minimum criteria	The graphic visualization will still be displayed if there is an error in the calculation of UC1 or in the system.

Table 7.3: Use case 3 evolved from the use case diagram as seen in figure 7.1.

7.2.1 Class diagrams

The class diagram based on the three use cases is illustrated in figure 7.2.

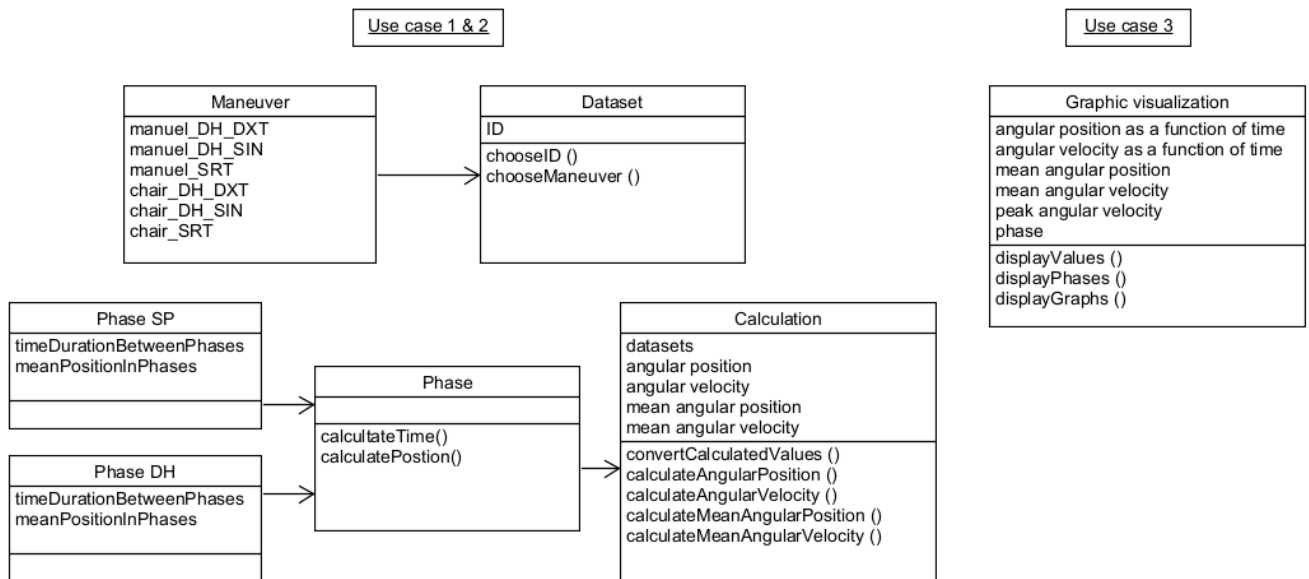


Figure 7.2: Class diagrams constructed on use case 1, 2 and 3.

7.3 Activity diagram

An activity diagram has been constructed to form an overview of the system's workflow, which is illustrated in figure 7.3. In this activity diagram the blue boxes represent the interactions the user makes whereas the green boxes represent the incorporated functions of the system.

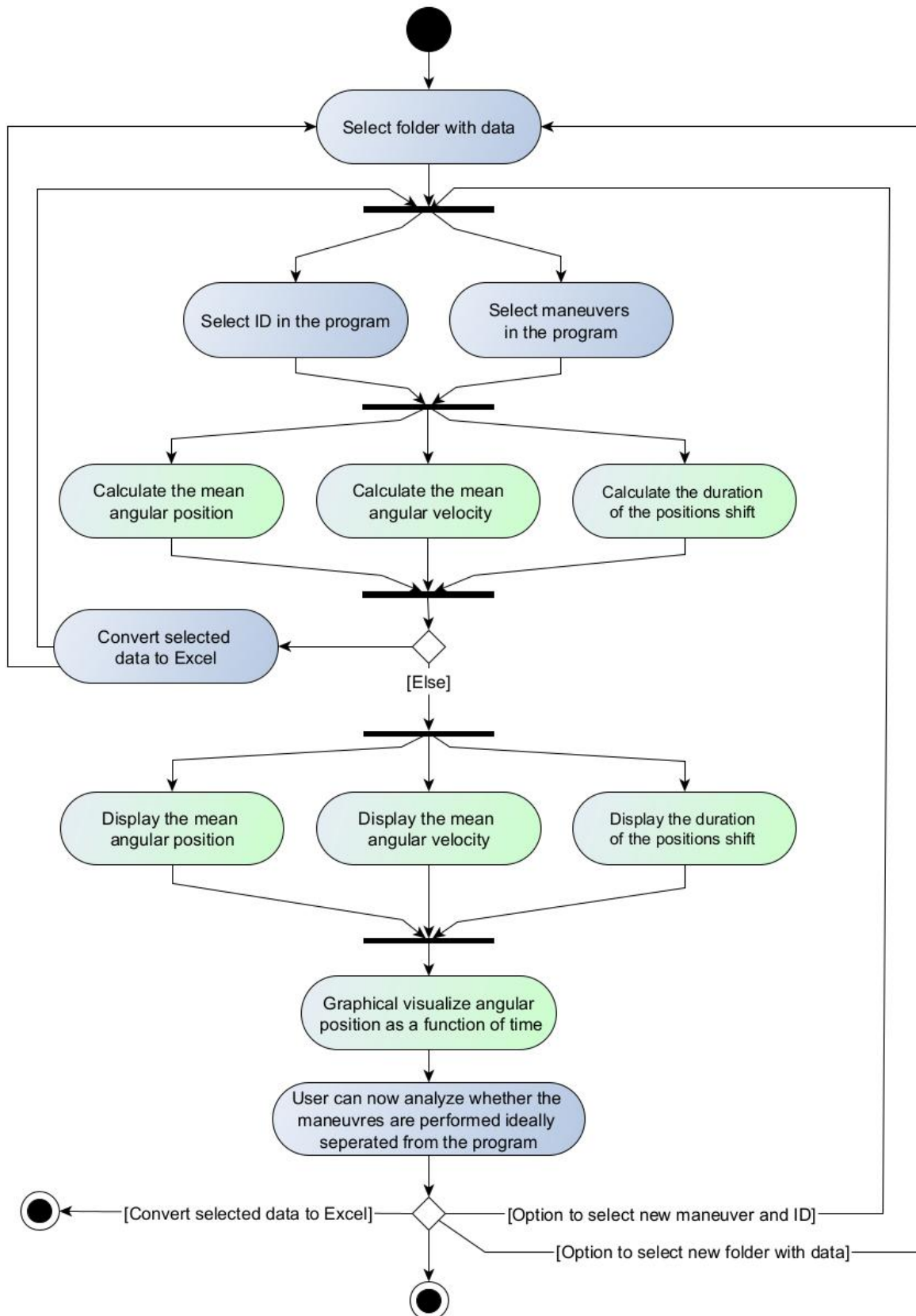


Figure 7.3: The activity diagram that illustrates the workflow of the system.

7.4 Prototype

The user scenario has been developed in collaboration with our external partner. The prototype is based on the user requirements, see chapter 6.1. Conversations and discussions have resulted in the creation of additional low-fidelity prototypes, these can be seen in appendix D. Figure 7.4 illustrates the fourth low-fidelity prototype that has been developed. The low-fidelity prototype is divided into steps to provide the user with a structured overview and a better understanding of how the system should be used in a specific order.

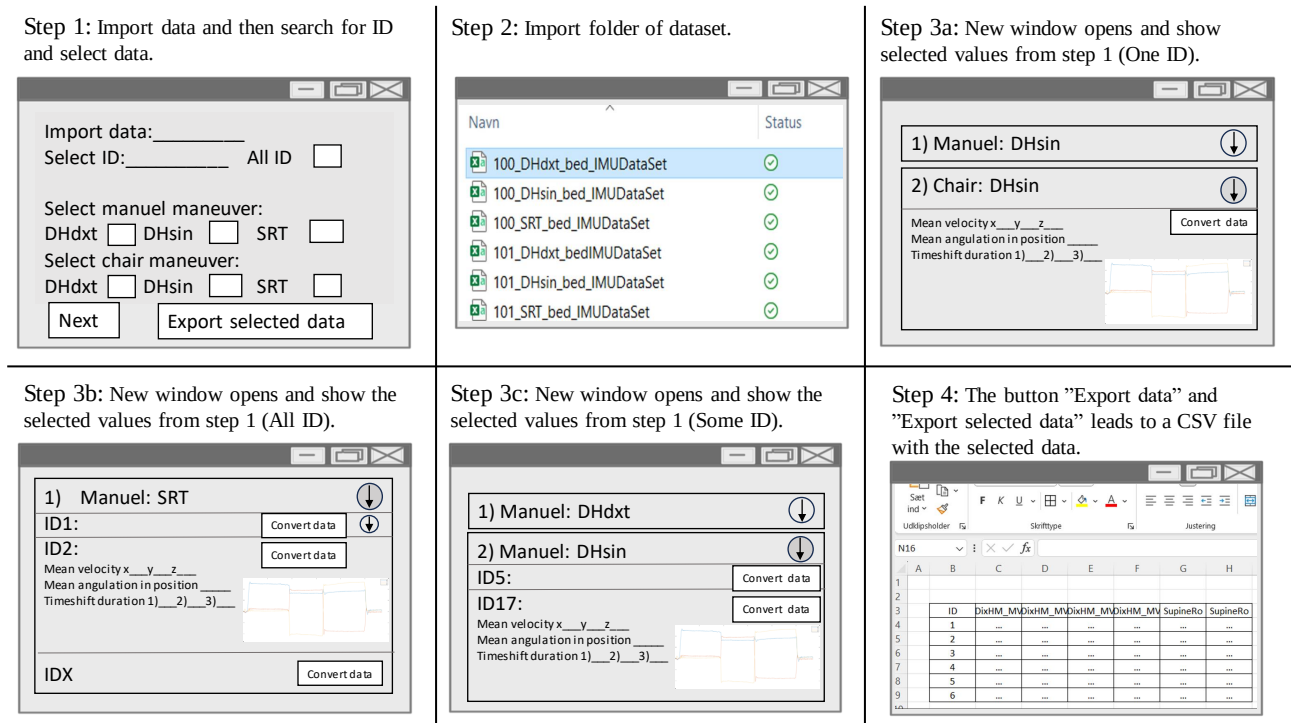


Figure 7.4: The fourth low-fidelity prototype. It shows in steps how to use the information system.

In 'Step 1' the user is provided with the option to import data, where step 2 will follow. In 'Step 2' the user selects the folder where the dataset is saved. The user can import as much or as little data as preferred. Once data has been imported for the first time, there's no need to repeat this process since it will be saved in the program.

After importing data the user has to choose the ID that the user wants to evaluate. The user can search for a specific ID, multiple IDs, or even all IDs simultaneously by clicking the box 'All ID'. Furthermore, the user should select the checkboxes corresponding to the maneuver they wish to view. The maneuvers in the program are abbreviated. The abbreviation is DHdxt for Dix-Hallpike right side, DHsin for Dix-Hallpike left side, and SRT for Supine Roll.

In 'Step 1', the user also has the option to export the calculated *values** of the selected data to an Excel file as seen in 'Step 4', by clicking 'Export selected data'. The user can also choose to click

'Next', leading to the appearance of a new window representing 'Step 3a-c'.

In 'Steps 3a-c' some different scenarios are shown based on the choices made in 'Step 1'. In the window, pop-out panels will appear that include the mean velocity, mean angular position, and time shift duration. Furthermore, in the pop-out panel, there will be a graphic visualization of the angular position as a function of time. The pop-out panel opens by clicking on the arrow for the maneuver and ID. 'Step a-c' are listed below.

- a: The user has selected one ID and two different maneuvers.
- b: The user has selected all IDs and one maneuver.
- c: The user has selected two IDs and two different maneuvers.

In each ID's pop-out panel, the user has the opportunity to export the *values** from the specific ID and maneuver by clicking 'Export data'.

In 'Step 4' the representation of exporting is shown when the user clicks 'Export selected data' in 'Step 1' or 'Export data' in 'Step 3'. The representation in the CSV file depends on the selected ID and maneuver. The ID colon will always be represented on the left side and the *values** for the maneuver can be read in a row corresponding to the ID.

8. System Requirements

An interpretation of the user requirements progresses to the subsequent phase in the V-Models verification phase, the system requirement specifications.

System requirements are a description of what the system should be able to perform, including interaction capabilities, data processing, and functionality. System requirements provide an understanding of the system and are critical for ensuring smooth operation of software applications. The system requirements have been developed based on user requirements.

8.1 Functional requirements

Based on the prototype and the user requirements the functional requirements have been developed. In table 8.1, the functional requirements are listed with their respective IDs and their relationship to the user requirements.

ID	Relation to user requirement-ID	Functional requirements	MoSCoW
FR1	UR1	The program must transform the quaternions to Euler angles.	Mo
FR2	UR2	The program must calculate, from the IMU datasets, the mean angular positions when the movements are static for the Supine Roll maneuver.	Mo
FR3	UR2	The program must calculate, from the IMU datasets, the mean angular positions when the movements are static for the Dix-Hallpike maneuver.	Mo
FR4	UR4	The program must calculate the mean angular velocity from the angular position during position shifts when the movements are dynamic for the Supine roll maneuver.	Mo
FR5	UR4	The program must calculate the mean angular velocity by the angular position during the position shifts, when the movements are dynamic for the Dix-Hallpike maneuver.	Mo
FR6	UR3	The program must calculate the time of the position shifts when the movements are dynamic for the Dix-Hallpike maneuver.	Mo
FR7	UR3	The program must calculate the time of the position shifts when the movements are dynamic for the Supine Roll maneuver.	Mo
FR8	UR2, UR3, UR4	The program must visualize the <i>values*</i> from UR2, UR3 and UR4 for the chosen maneuvers.	Mo
FR9	UR6	The program must include graphs of the head angular position as a function of time.	S
FR10	UR7	The program must be able to convert chosen <i>values*</i> from UR2, UR3 and UR4 <i>values*</i> .	Co

Table 8.1: The functional requirements listed in MoSCoW prioritization.

8.2 Non-functional requirements

In table 8.2, the nonfunctional requirements based on the user requirements and the prototype, are listed. These are the conditions the system should operate under, as well as the quality of the system.

ID	Non-functional requirements	MoSCoW
NFR1	The system must address the imported data in accordance with the General Data Protection Regulation (GDPR) rules.	Mo
NFR2	The system must be compatible so it can communicate with file explorer.	Mo
NFR3	The system must be compatible with a computer with Java installed.	Mo
NFR4	The system must be compatible so it can communicate with CSV files.	Mo
NFR5	The system must have a maximum step of 5 clicks to access the desired data.	S
NFR6	The system must calculate and display the desired <i>values*</i> within 10 seconds.	S
NFR7	The system must be flexible so future functions can be added.	Co

Table 8.2: The non-functional requirements listed in MoSCoW prioritization.

Part IV

Software development

9. Software development

In the following sections, the final design of the prototype has been developed and presented. Furthermore, the process and the description of transitioning from system design to implementation will be clarified. During the implementation phase, High level Design, Low level Design, and coding from the V-Model's verification phase will be employed. These three stages collectively provide an understanding of the system composition, an explanation of the designed classes, and how these classes interact with each other.

9.1 Object-oriented programming implementation

Based on the diagrams and methods formulated in the OOA- and OOD process, the system has been made through the program Apache NetBeans IDE 18. The purpose of the program is to assist developing different Java applications, as NetBeans IDE is written to support Java [40]. The system is intended for an external partner, and therefore the program should be possible to pass without the installation of Apache NetBeans IDE 18. A compiled Java code can run on all supported Java platforms, without recompiling, which makes it possible to convey. Java is an OOP language, which is class-based, therefore it follows the OOP method. Through the development of the system, the focus has been on the implementation of OOP three basic ideas: objects, messages, and classes, as described in section 5.5.[41]

Related to the implementation of OOP, a new class diagram has been evolved. The new class diagram is a reassessment of the class diagram from OOD shown in figure 7.2. Through the OOM process, the class diagram has been more specified and reflects how the class is represented in the code whereas the class diagram from OOD reflects the low-fidelity prototype. The class diagram in OOP illustrates the classes used in the program and how they are related to each other. In the class diagram, the different methods and attributes are represented. The class diagram is illustrated in figure 9.1.

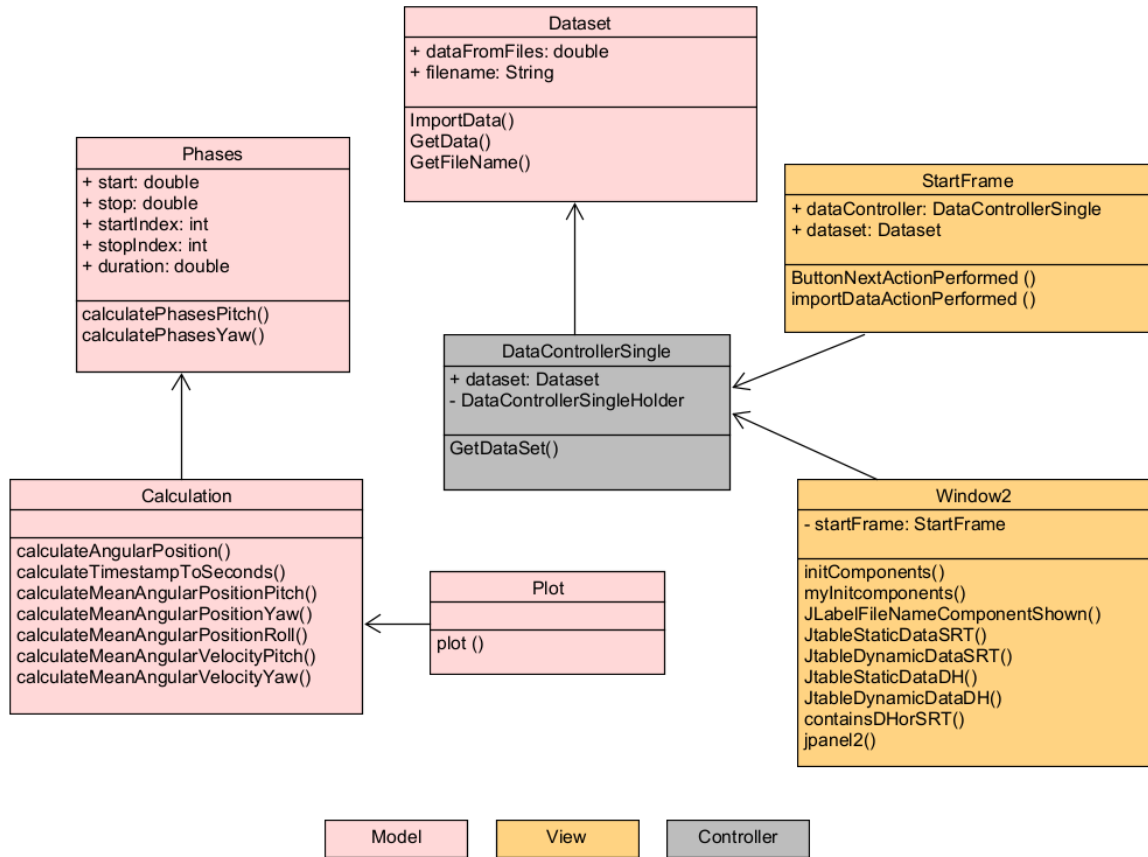


Figure 9.1: Illustration of the class diagram in OOP divided into model, view, and controller.

As seen in the class diagram in figure 9.1, the pattern MVC is used, as described in section 5.5. The 'Calculation' is a model class, which calculates the data that is used in the 'Window2' class to display the data in the interface, which makes it a view class. The 'DataControllerSingle' is a controller class, which maintains the uploaded datasets.

In addition to the MVC pattern, various other design patterns have been employed. They are expressed in several places in the code. For example, the 'DataControllerSingle' class employs the singleton design pattern. As described in section 5.5, the singleton pattern makes sure that a class has only one instance with global access to a singular occurrence. The singleton pattern in the 'DataControllerSingle' class ensures that only one dataset can be uploaded at a time. After the calculations for the imported dataset have been displayed in 'Window2', a new dataset can be uploaded. This implies that after the system has been executed, the 'DataControllerSingle' can create a new instance for a new dataset.

Related to the implementation of OOP, a new activity diagram has been prepared. In connection with the class diagram adjustments, the activity diagram's flow has been reviewed and changed. The new activity diagram is a reassessment of the activity diagram from OOD shown in figure 7.3. The new

activity diagram illustrates how the different activities are related to each other and an overview of the workflow is implemented in the program.

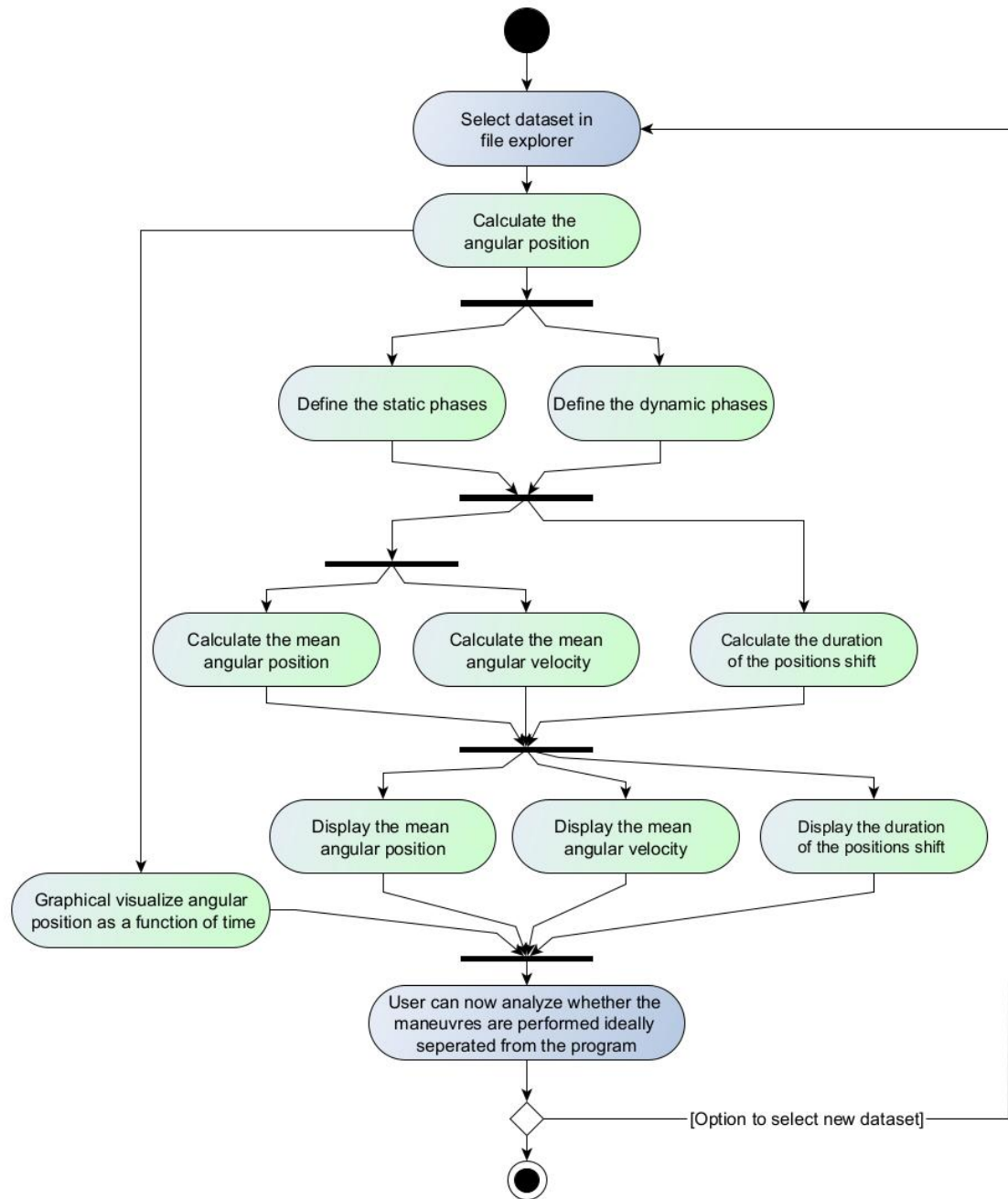


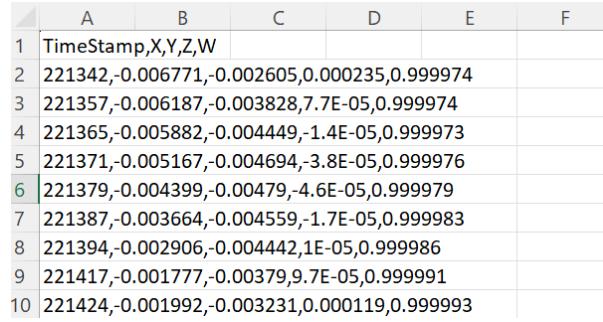
Figure 9.2: Illustration of the activity diagram prepared in OOP.

Based on the evolved OOP diagrams the program has been developed. The class and the activity diagram can be used for further implementation and development. In the following sections, the software is described to give insight into how the classes and methods are used in the system, as seen in the class diagram, figure 9.1.

9.2 Design

In the program, it's essential to import data since the data is used in the calculations. The purpose of importing the data is to meet the requirements NFR2 and NFR4, see section 8.2.

Datasets have been provided by our external partner at the first meeting. These datasets contain data obtained from an IMU sensor represented in the form of quaternions, as explained in section 3.5 and visualized in appendix B. Additionally, the datasets have been provided in the format of CSV files. Each dataset has a filename that consists of a patient-specific ID and the name of the maneuver. The maneuver is abbreviated DH for Dix-Hallpike and SRT for Supine roll. Dix-Hallpike can be executed to the left or the right, whereas the right is abbreviated dxt and the left is abbreviated sin. Each dataset consists of the values timestamp, x, y, z, and w. The values for x, y, z, and w are rotations which are expressed as comma-separated values. Below is a picture of how the raw data is represented by opening the CSV file in Microsoft Excel.



	A	B	C	D	E	F
1	TimeStamp,X,Y,Z,W					
2	221342,-0.006771,-0.002605,0.000235,0.999974					
3	221357,-0.006187,-0.003828,7.7E-05,0.999974					
4	221365,-0.005882,-0.004449,-1.4E-05,0.999973					
5	221371,-0.005167,-0.004694,-3.8E-05,0.999976					
6	221379,-0.004399,-0.00479,-4.6E-05,0.999979					
7	221387,-0.003664,-0.004559,-1.7E-05,0.999983					
8	221394,-0.002906,-0.004442,1E-05,0.999986					
9	221417,-0.001777,-0.00379,9.7E-05,0.999991					
10	221424,-0.001992,-0.003231,0.000119,0.999993					

Figure 9.3: Example of a dataset in the CSV file format opened in Excel.

9.2.1 Data class

In the 'Dataset' class, the procedure for importing the dataset into the program is defined. This class is made to use the file explorer on the computer to make the program flexible for other users. The Java-specific method 'JFileChooser' is used which makes it possible to open the file explorer whereas the method 'getSelectedFile' allows to select a file in the file explorer. The selected name of the file is saved as 'filename' for later use in the program.

A 'try/catch' construction is used in case an error occurs. In the 'try/catch' block, it's specified that it's a CSV file intended to be read into the program. Furthermore, it's also specified that the first row in the CSV file shouldn't be read since it's the headerline. An array is created to save the data from each row after the array of data is called 'dataFromFiles'.

In the class 'StartFrame' the GUI toolkit 'javax.swing' is used. Using 'javax.swing' it's possible to create desktop applications in Java. It enables the user to create a user interface that matches the purpose of the program.

The file explorer will open when the button 'Import data' is pressed and the user can select the desired file from the file explorer on the computer. The name of the selected file will be written instead of 'No file selected'. The 'StartFrame' GUI is visualized in figure 9.4 and 9.5. Figure 9.4 shows that no file is selected. Figure 9.5 shows when a file is selected.

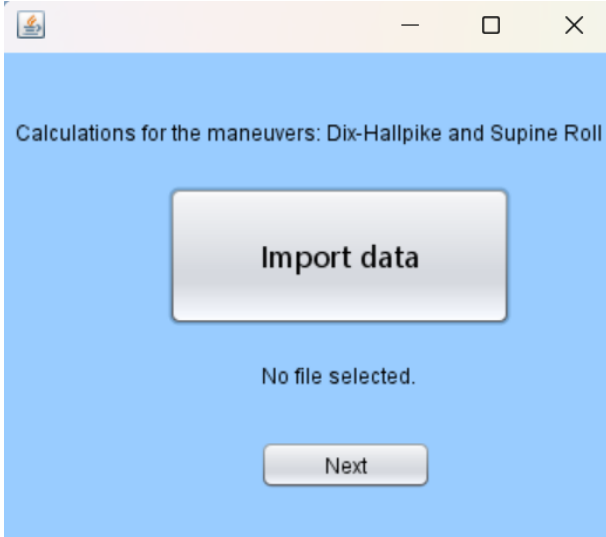


Figure 9.4: A visualization of the GUI of 'StartFrame' in the program without a selected file.

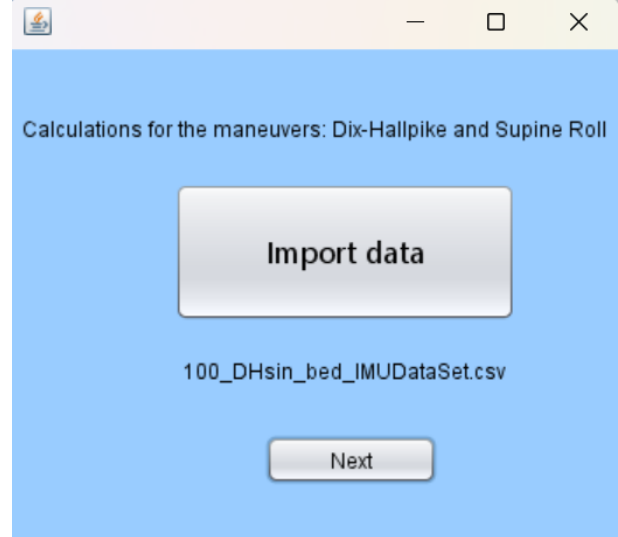


Figure 9.5: A visualization of the GUI of 'StartFrame' in the program with a selected file.

The class 'Window2' will open by clicking the button 'Next' in 'StartFrame' where 'javax.swing' also is used. At the top of 'Window2', the filename of the selected file will be displayed. The tables in 'Window2' are depending on the filename. In the class 'Window2' there is created an 'if-statement' which determines whether there is DH or SRT in the file name, which indicates whether it's a Dix-Hallpike- or Supine roll maneuver. Depending on the filename the belonging tables will be displayed in 'Window2'. The tables consist of the different calculations depending on the selected file. A table with static and dynamic phases is displayed, this is described further in section 9.2.2.

'Window2' will visualize a plot of the maneuver depending on the selected file. If a file name without SRT or DH is selected, an empty table is displayed due to the calculations only fit the specific files. Furthermore, the plot will be empty in 'Window2' since there is nothing to plot. The plot is defined in a class called 'Plot'. In 'Plot' is 'dataFromFiles' used which refers to the imported data. Based on the imported data, the three dimensions of pitch, yaw, and roll are plotted in the same graph. The graph will visualize degrees as a function of time in seconds for the three dimensions and these are visualized with different colors.

In figure 9.6 'Window2' is visualized where tables and a graph are displayed. The 'Window2' is executed from the final prototype, as seen in appendix D.

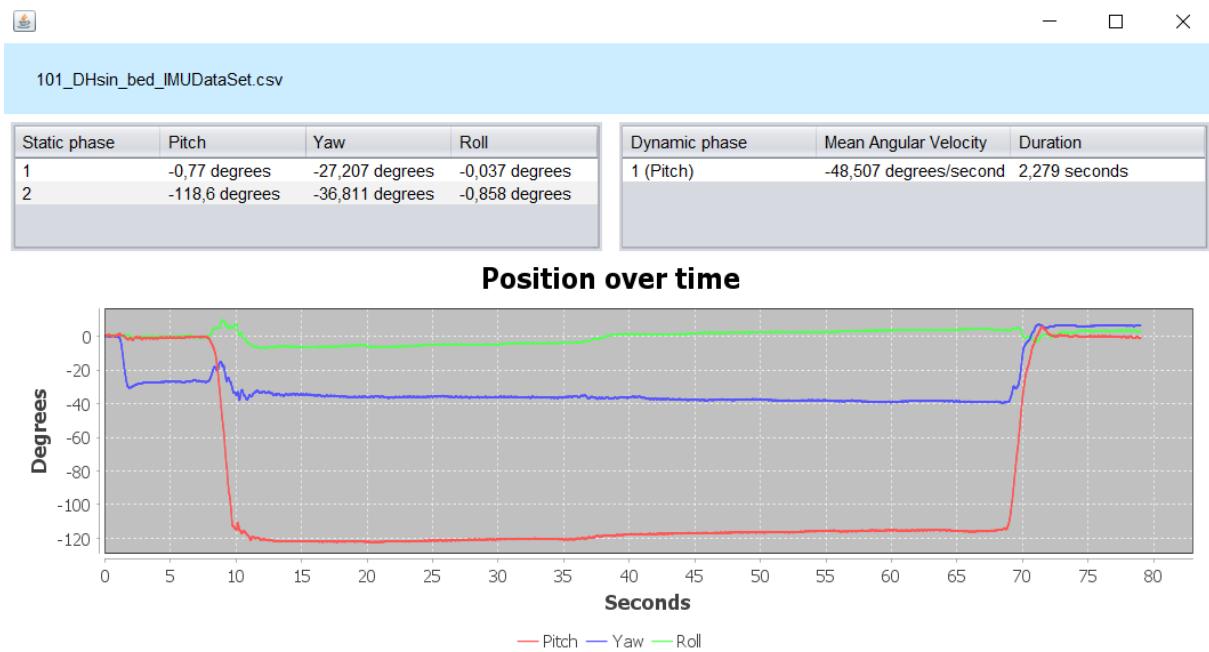


Figure 9.6: A visualization of the GUI of 'Window2' in the program.

9.2.2 Calculation- and Phases class

To meet the functional requirement for FR1, it's necessary to transform the data from quaternions to Euler angles. This is an important step as the fulfillment of functional requirements FR2, FR3, FR4, FR5, and FR6 depends on the calculation. The functional requirements can be seen in table 8.1. These requirements are achieved by declaring position shifts. To find the position shifts, the maneuvers have been divided into phases, static- and dynamic phases.

The raw data that was provided from the datasets, is given in quaternions. To convert quaternions into Euler angles, an intermediate step is employed. This step is to convert the quaternions to rotation matrices as illustrated in figure 9.7. The purpose of calculating the rotation matrices is to determine the rotation about the three dimensions x,y, and z. Depending on, the rotation is about x,y, or z, different angles describe the rotation of the three axes. These angles are described as the Euler angles, around which the rotation occurs first. As seen in figure 9.8 the order of the rotation, in the calculation for the positions, is ZYX. From the rotation matrix, the Euler angles for each pitch, yaw, and roll can be calculated. [42]

$$R(Q) = \begin{bmatrix} 2(q_0^2 + q_1^2) - 1 & 2(q_1q_2 - q_0q_3) & 2(q_1q_3 + q_0q_2) \\ 2(q_1q_2 + q_0q_3) & 2(q_0^2 + q_2^2) - 1 & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_2q_3 + q_0q_1) & 2(q_0^2 + q_3^2) - 1 \end{bmatrix}$$

Figure 9.7: The rotational matrix used for the calculation of pitch, yaw, and roll.[20]

The method 'calculateAngularPosition', as seen in figure 9.8 is used to first create the rotation matrices using the quaternions. Here q represents an array of components of the quaternion. Next, the method 'getAngles' returns the Euler angles in radians from x,y and z. Last, the method 'Math.toDegrees' is used to convert Euler angles in radians to Euler angles degrees.

```
/*
 * Quaternion is converted to Euler angles to determine the angular position for each example.
 * The Calculation is based on the raotation matrix for the various quaternions.
 */
public static double[] calculateAngularPosition(double[] q) {
    Rotation rotation = new Rotation(q[4], q[1], q[2], q[3], needsNormalization: false);
    double[] eulerAngles = rotation.getAngles(order: RotationOrder.ZYX);
    for (int i = 0; i < eulerAngles.length; i++) {
        eulerAngles[i] = Math.toDegrees(eulerAngles[i]);
    }
    return eulerAngles;
}

public static double calculateTimestampToSeconds(double starttime, double timestamp) {
    return (timestamp - starttime) / 1000;
}
```

Figure 9.8: The method converting quaternions to Euler angles in degrees.

After converting the Euler angles to degrees, the next step is to categorize the phases. In the static

phases, the head should remain in the same position and have minimal changes in degrees. In the dynamic phases, the head was in motion, which indicates that the patient's head is shifting to a new position.

The phases are indicated by a threshold as shown in figure 9.9. The system will find a position shift if pitch or yaw changes by more than 13 degrees within 120 samples. The thresholds are the same for the Supine Roll maneuver, Dix-Hallpike maneuver left, and Dix-Hallpike maneuver right. After declaring the threshold there is a for-loop looking through all samples and finding where the thresholds are obtained.

```
public static List<Faser> calculatePhasesPitch(double[][] euler, double[] timeSeconds) {
    double pitchThresholdDegrees = 13;
    int pitchThresholdSamples = 120;
    double acceptDuration = 0.5;
```

Figure 9.9: A part of the script that declares the phases. It shows how the threshold is implemented in the program.

Figure 9.10 represents the maneuver of a Dix-Hallpike to the left. The yaw vector is used to indicate static phase 1, this is where the patient's head is turned 45°. In this phase, the class 'Calculation' finds the mean angular position for pitch, yaw, and roll. In dynamic phase 1 the patient is shifted to a supine position and the mean angular velocity and the duration will be calculated. The pitch vector is used to indicate static phase 2 where the mean angular position is calculated for the pitch, yaw, and roll. In this phase, the patient is laid in a supine position at 120 degrees.

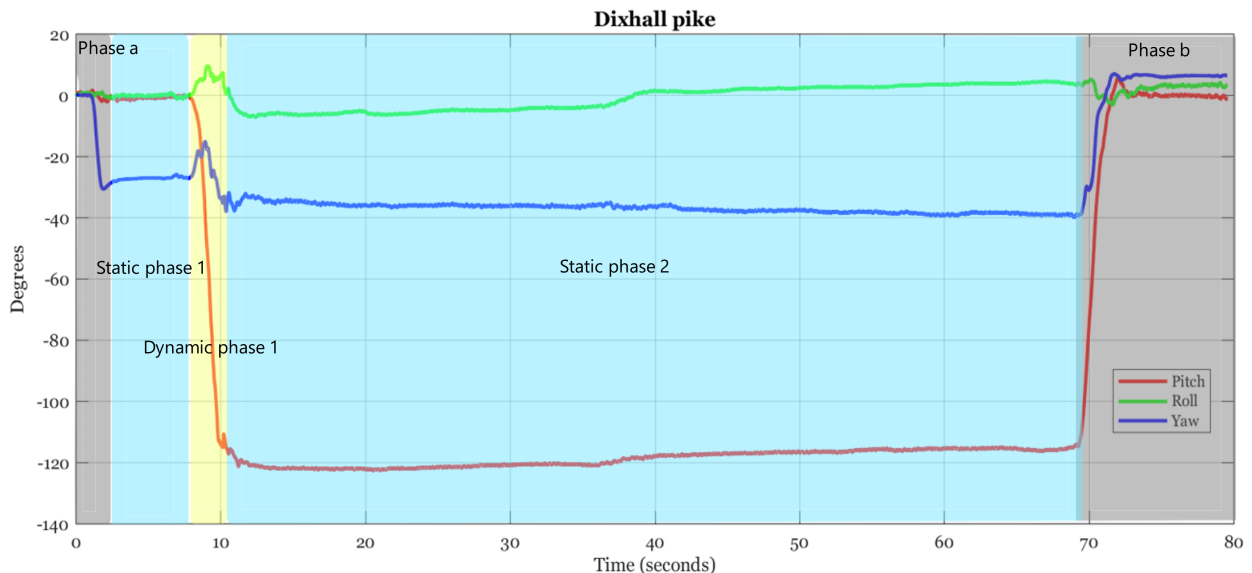


Figure 9.10: The phases for a Dix-Hallpike left are declared both for the static and dynamic phases.

Figure 9.11 represents a graph for a Supine Roll maneuver. Phases a and b are deemed unimportant for the user, and therefore, no calculations are performed for these phases. In the static phases 1, 2, and 3 the head remains in the same position and therefore it's possible to calculate the mean angular

position. In the dynamic phase 1, 2, and 3 the patients head is in motion, and from these phases the mean angular velocity is calculated and the duration in seconds.

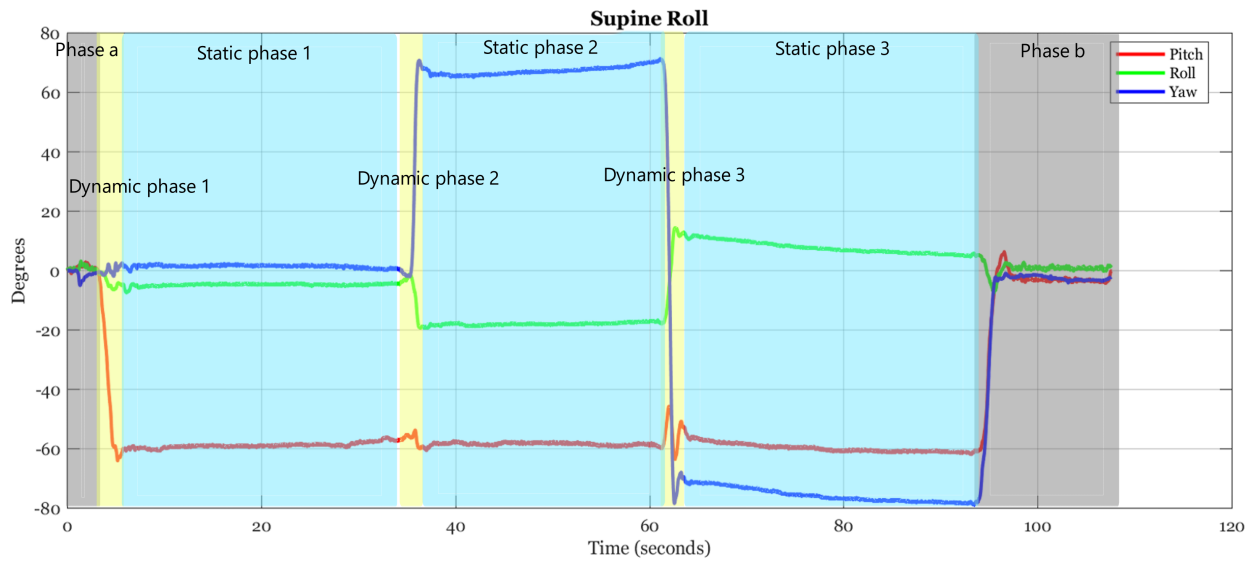


Figure 9.11: The phases for a Supine Roll maneuver are declared, which have static and dynamic phases.

From the static phases, the mean angular positions are calculated and from the dynamic phases, the mean angular velocities and duration are calculated.

The figure 9.12 displays a section of the code to calculate the mean angular position for pitch in the static phase. It's designed based on the same concept for both yaw and roll. The mean angular positions are calculated by declaring a start phase and a stop phase. The for-loop count is the index amount between the start phase and the stop phase. Afterward, the mean angular position is calculated by subtracting 'startIndex' from 'stopIndex' and then dividing the result by the sum of the pitch angles.

```

/*
 * The next functions calculate the mean angular position during static phases.
 * Based on the calculated angles and defined phases (class Phases)
 */
public static double calculateMeanAngularPositionPitch(Faser fase1, Faser fase2, double[][] euler) {
    int startIndex = fase1.stopIndex;
    int stopIndex = fase2.startIndex;
    double sum = 0;

    for (int i = startIndex; i <= stopIndex; i++) {
        sum += euler[i][2];
    }

    double mean = sum / (stopIndex - startIndex + 1);

    return mean;
}

```

Figure 9.12: Section of the code to calculate the mean angular position for pitch.

Figure 9.13 displays a section of the code to calculate the mean angular velocity for pitch in the dynamic phase. It's designed based on the same concept for yaw. The mean angular velocity is calculated by dividing 'deltaAngle' with 'deltaTime'. 'deltaAngle' calculates the difference between the Euler angle at the current and previous indexes. 'deltaTime' calculates the time difference between the current and previous indexes.

```

/*
 * The next 2 calculations determine the angular velocity during dynamic phases
 * based on the calculated angles, phases, and time.
 */
public static double calculateMeanAngularVelocityPitch(Faser faser, Faser stopFaser, double[][] euler, double[] time) {
    int startIndex = faser.startIndex;
    int stopIndex = faser.stopIndex;
    double sum = 0;
    int validCount = 0;

    for (int i = startIndex + 1; i <= stopIndex; i++) {
        if (time[i] != time[i - 1]) {
            double deltaAngle = euler[i][2] - euler[i - 1][2];
            double deltaTime = time[i] - time[i - 1];
            double angularVelocity = deltaAngle / deltaTime;
            sum += angularVelocity;
            validCount++;
        }
    }
    double meanAngularVelocity = (validCount > 0) ? sum / validCount : 0;
    return meanAngularVelocity;
}

```

Figure 9.13: Section of the code to calculate the mean angular velocity for pitch.

In the datasets, timestamps are in milliseconds, while velocity and duration are calculated in seconds in the dynamic phases. This is achieved by the method 'calculateTimestampToSeconds'. The method computes the time difference between 'timestamp' and 'starttime' and divides this result by 1000 to convert milliseconds to seconds.

9.3 Structure and class interactions

To illustrate the structure and interaction between the classes in the program, a sequence diagram has been developed, which is visualized in figure 9.14. The sequence diagram illustrates when the different classes with related methods are used. The diagram shows the flow of the classes in the program and when they are active.

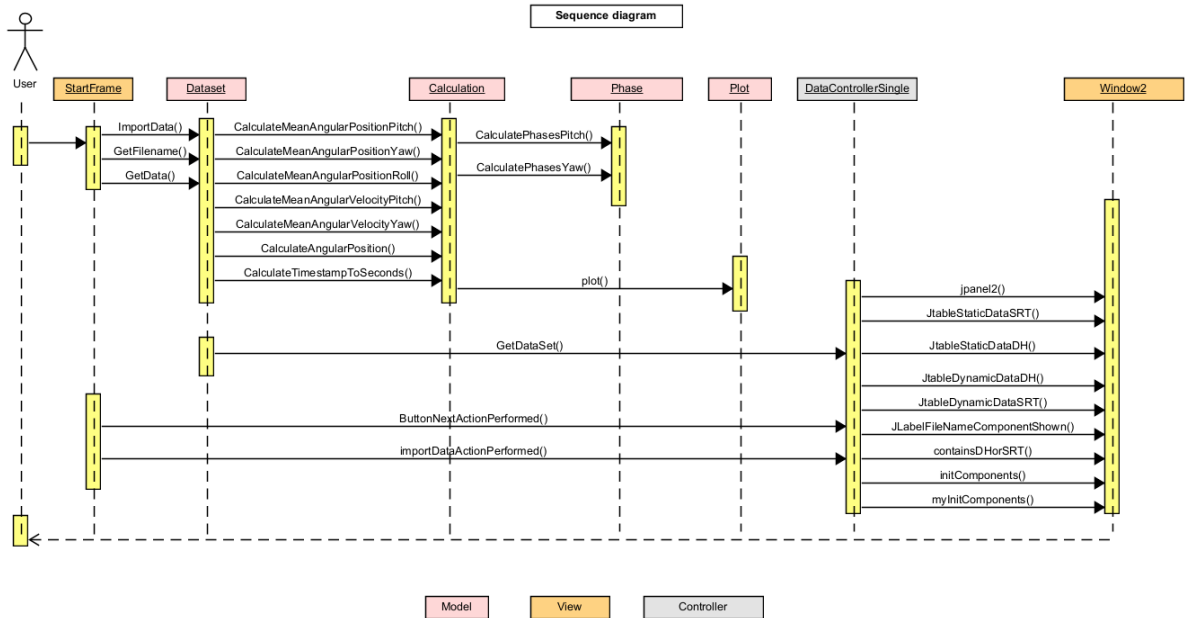


Figure 9.14: Illustration of the sequence diagram prepared after the software development.

Figure 9.15 illustrates the interaction between the view class 'StartFrame' and the model class 'Dataset', along with the controller class 'DataControllerSingle'. This depiction outlines the overarching structure of the software.

```

import Data.DataControllerSingle;
import Data.Dataset;

public class StartFrame extends javax.swing.JFrame {

    DataControllerSingle dataController;
    Dataset dataset;

    public StartFrame() {
        initComponents();

        dataController = DataControllerSingle.getInstance();
        dataset = dataController.GetDataSet();
    }
}
  
```

Figure 9.15: Section of software illustrating how classes are imported in the class 'startFrame' and defined in the constructor.

Part V

Test

10. Test of the program

To test the program, system integration and unit testing have been carried out. Unit testing has been used to test whether the program performs as expected based on the functional and non-functional requirements, as seen in chapter 8. This ensures that each requirement has been achieved by testing smaller units at a time. The unit test is described in section 5.1. Following the system integration testing, a user acceptance test has been performed. The external partner acceptance testing involves having the intended users try out the program without help. Afterwards, an interview has taken place.

10.1 Test of FR1

In the test of FR1 the V-Model is used, in the validation phase here below the unit testing is applied. Unit testing is used as there is a need to test on a single unit as in FR1.

The purpose of testing FR1 is to test whether the program is able to calculate and convert from quaternions to Euler angles.

ID	Relation to user requirement-ID	Functional requirements
FR1	UR1	The program must transform the quaternions to Euler angles.

Procedure:

1. Open the program, press the 'Import data' button and select a dataset.
2. The calculated values get printed in the output window as Euler angles.

Results:

The test can be performed for both Supine Roll and Dix-Hallpike positions, both left and right. By importing the data in quaternions, the script returns the values in Euler angles, as seen in figure 10.1, for every sample in the datasets.

```
[56.03477441686939, -11.905307284801559, -111.53157392185959]
[56.00437732004429, -11.917540521171308, -111.56789512310868]
[55.95726841629882, -11.939941490825733, -111.59150596650943]
[55.912999754058006, -11.964098319465492, -111.61449893193661]
[55.91658084403096, -11.974277566172406, -111.66314371458833]
```

Figure 10.1: A section of the printed Euler angles in the output window.

Conclusion:

Based on the results of the test, it can be concluded that requirement FR1 have been obtained and it's possible to transform data from quaternions to Euler angles.

10.2 Test of Dix-Hallpike

To test the functional requirements of FR3, FR5, FR6, FR8, and FR9 component testing is applied. Component testing is a method in the validation phase for the V-model. Component testing is used as it covers a single component or module at a time.

The purpose of testing the Dix-Hallpike maneuver is to test whether the values for Dix-Hallpike can be calculated and visualized.

ID	Relation to user requirement-ID	Functional requirements
FR3	UR2	The program must calculate, from the IMU datasets, the mean angular positions when the movements are static for the Dix-Hallpike maneuver.
FR5	UR4	The program must calculate the mean angular velocity by the angular position during the position shifts, when the movements are dynamic for the Dix-Hallpike maneuver.
FR6	UR3	The program must calculate the time of the position shifts when the movements are dynamic for the Dix-Hallpike maneuver.
FR8	UR2, UR3, UR4	The program must visualize the <i>values*</i> from UR2, UR3 and UR4 for the chosen maneuvers.
FR9	UR6	The program must include graphs of the head angular position as a function of time.

Procedure:

1. Open the program, press the 'Import data' button and select a dataset of a Dix-Hallpike maneuver.
2. Press the 'Next' button, tables and a graph will be visualized in 'Window2'.

Results:

The test is performed for both Dix-Hallpike left and Dix-Hallpike right. The test for Dix-Hallpike right is visualized in figure 10.2. The same outcome applies to the test of Dix-Hallpike left. The result of FR3 test indicates that the mean angular positions are being calculated when the movements are static. This is illustrated in 'Window 2' in the table on the left side of figure 10.2. The table on the left side shows the two static phases of the Dix-Hallpike maneuver where the angular positions for pitch, yaw, and roll are calculated for each phase.

Similarly, the FR5 test indicates that the mean angular velocity is being calculated during the dynamic phases. This is also depicted in figure 10.2, where the table on the right side illustrates the dynamic

phase. Here, dynamic phase 1 is calculated for pitch. Additionally, FR6 is obtained from the table of the dynamic phase, showing the duration in seconds.

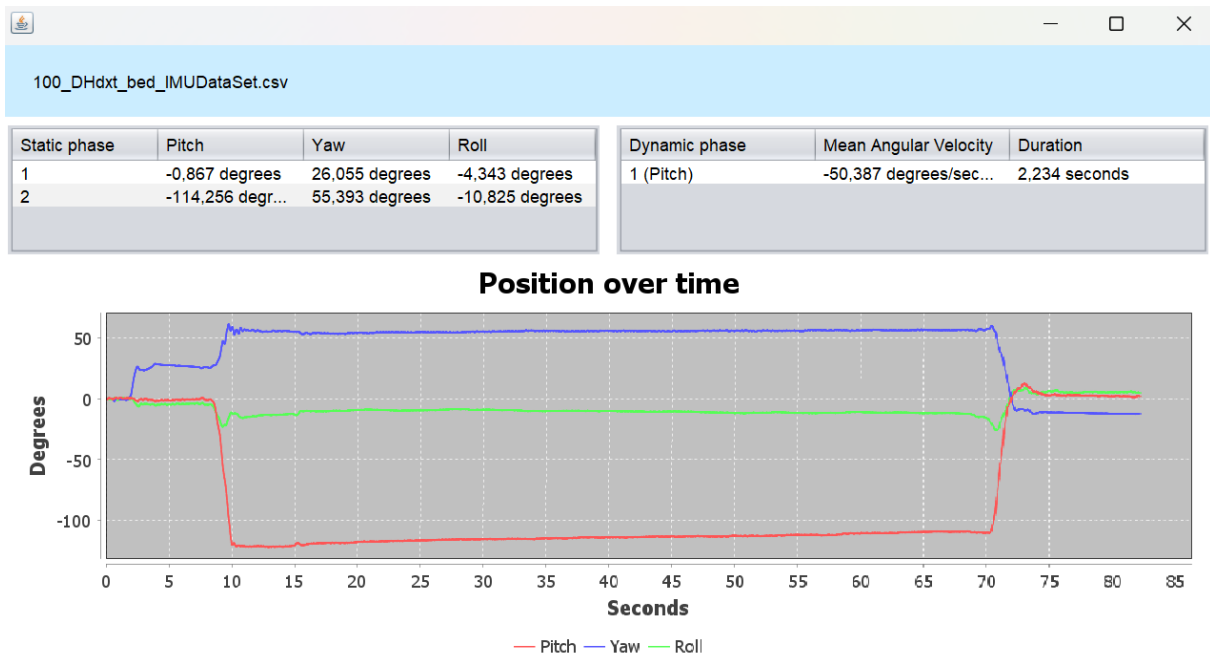


Figure 10.2: A dataset of Dix-Hallpike maneuver right side is tested in the program. The calculations of the mean angular position are shown in the table to the left and the mean angular velocity and duration are shown in the table to the right. A graph of the position over time is plotted in the lower section of the window.

As previously elucidated, the mean angular position in the static phases, the mean angular velocity, and the duration of the dynamic phases are visualized in 'Window2', as seen in figure 10.2. Therefore, requirement FR8 is obtained. At the lower section of 'Window2', a graph of the head angular position is plotted as a function of time; thus, requirement FR9 is obtained.

Conclusion:

Based on the results of the test, it can be concluded that all requirements related to the Dix-Hallpike maneuver have been obtained.

10.3 Test of Supine Roll

To test the functional requirements of FR2, FR4, FR7, FR8, and FR9 component testing is applied. Component testing is a method in the validation phase for the V-model. Component testing is used as it covers a single component or module at a time.

The purpose of testing the Supine Roll maneuver is to test whether the values for Supine Roll can be calculated and visualized.

ID	Relation to user requirement-ID	Functional requirements
FR2	UR2	The program must calculate, from the IMU datasets, the mean angular positions when the movements are static for the Supine Roll maneuver.
FR4	UR4	The program must calculate the mean angular velocity from the angular position during position shifts when the movements are dynamic for the Supine roll maneuver.
FR7	UR3	The program must calculate the time of the position shifts when the movements are dynamic for the Supine Roll maneuver.
FR8	UR2, UR3, UR4	The program must visualize the <i>values*</i> from UR2, UR3 and UR4 for the chosen maneuvers.
FR9	UR6	The program must include graphs of the head angular position as a function of time.

Procedure:

1. Open the program, press the 'Import data' button and select a dataset of a Supine Roll maneuver.
2. Press the 'Next' button, tables and a graph will be visualized in 'Window2'.

Results:

The results of the FR2 test indicate that mean angular positions are being calculated when the movements are static. This is illustrated in 'Window 2' in the tables at the top left of the figure 10.3. The left table shows the three static phases of the Supine roll maneuver, where pitch, yaw, and roll are calculated for each phase.

Similarly, the FR4 test indicates that the mean angular velocity is being calculated during the dynamic phases. This is also depicted in Figure 10.3, where the table on the right illustrates the dynamic phase. Here, dynamic phase 1 is calculated for pitch, and dynamic phases 2 and 3 are calculated for yaw. Additionally, FR6 is obtained from the same table, showing the duration of the dynamic phases in seconds.

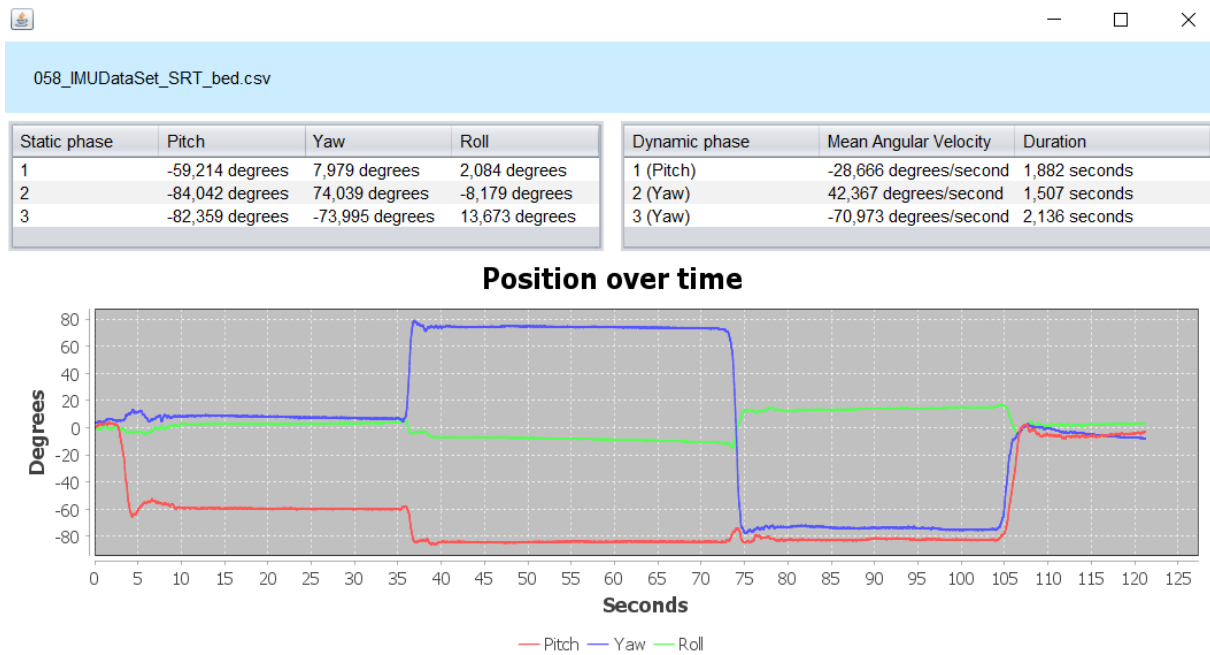


Figure 10.3: A dataset of the Supine Roll maneuver is tested in the program. The calculations of the mean angular position are shown in the table to the left and the mean angular velocity and duration are shown in the table to the right. A graph of the position over time is plotted in the lower section of the window.

As previously explicated, the mean angular position in the static phases, the mean angular velocity, and the duration of the dynamic phases are visualized in 'Window2', as seen in figure 10.3. Therefore, the FR8 is obtained. At the lower section of 'Window2', a graph of the head angular position is plotted as a function of time; thus, requirement FR9 is obtained.

Conclusion:

Based on the results of the test, it can be concluded that all requirements related to the Supine Roll maneuver have been obtained.

10.4 Test of non-functional requirements

In this section, non-functional requirements are tested. Unit testing is employed for the tests of NFR2, NFR4, and NFR5, while integration testing is used for NFR3. These methods are applied during the validation phase of the V-Model. NFR6 involves benchmarking, and NFR7 employs OOM.

10.4.1 NFR1

ID	Non-functional requirements
NFR1	The system must address the imported data in accordance with the General Data Protection Regulation (GDPR) rules.

The General Data Protection Regulation (GDPR) is a law whose purpose is to protect personal data in the EU. The GDPR law is divided into categories such as non-sensitive data and sensitive data.

Sensitive data is confidential information and an example of this could be a CPR number. The non-sensitive data can be pseudonymized or anonymized. Pseudonymized data means a specific person can't be identified without the use of additional data. Anonymized data is data there on its own or in combination with other data can't be traced back to the person. Anonymization should be irreversible and GDPR is in this case not present. The GDPR law is used if data can be led back to a person to the extent that the person can be identified.[43] The non-functional requirement NFR1 is obtained since the imported data solely denotes the patient ID and name of the maneuver, precluding any direct traceability to the patient which means the data is anonymized. Therefore, the program addresses the imported data in accordance with the GDPR rules.

10.4.2 NFR2

ID	Non-functional requirements
NFR2	The system must be compatible so it can communicate with file explorer.

In the program, upon selecting 'Import data,' the file explorer specific to the respective computer appears, as seen in figure 10.4, from which the desired data can then be uploaded, which obtained requirement NFR2.

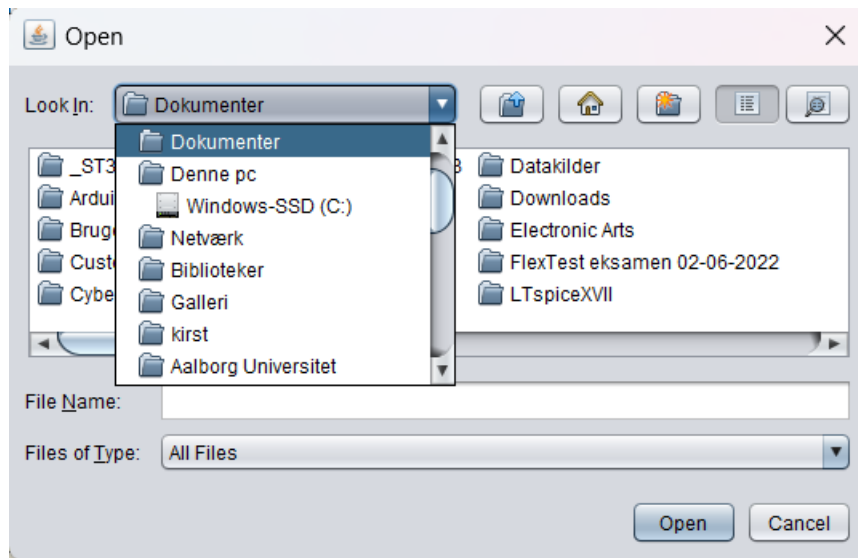


Figure 10.4: A file explorer for a specific PC appears, allowing the selection of a file.

10.4.3 NFR3

ID	Non-functional requirements
NFR2	The system must be compatible with a computer with Java installed.

The program is exported as a JAR file which necessitates that the program must be compatible with a computer equipped with Java, thereby requirement NFR3 is obtained.

10.4.4 NFR4

ID	Non-functional requirements
NFR4	The system must be compatible so it can communicate with CSV files.

Attempting to import files other than a CSV file, causes 'Window2' is not displayed, indicating an error has occurred. The error messages in the output specify that only CSV files are permissible for import, ensuring the prevention of errors during the data import process. By this, requirement NFR4 is obtained.

10.4.5 NFR5

ID	Non-functional requirements
NFR5	The system must have a maximum step of 5 clicks to access the desired data.

The program only takes 3 clicks to access the desired data, which obtains requirement NFR5. The initial click is on the 'Import data' button. This opens the interface for uploading the desired data. Subsequently, the second click is on open, to upload the desired data. The final click on the 'Next' button which opens 'Window2', where the data is presented without the need for further interaction.

10.4.6 NFR6

ID	Non-functional requirements
NFR6	The system must calculate and display the desired <i>values</i> * within 10 seconds.

To test the requirement NFR6, the methodology known as benchmarking is employed. Benchmarking is a testing procedure employed on a computer system to assess its performance, such as evaluating the execution time of the code [44]. The benchmark test on 'Window2' was performed, when uploading one dataset, to assess the duration of the execution time upon clicking the 'Next' button. Following the execution, the elapsed time concluded at 21 milliseconds, thereby satisfies the requirement NFR6, which specifies that values must be displayed within 10 seconds.

10.4.7 NFR7

ID	Non-functional requirements
NFR7	The system must be flexible so future functions can be added.

To facilitate the attainment of the requirement NFR7 the method OOM is used. Due to the incorporation of OOM during the development of the program, as detailed in section 5.2, the application of this method not only facilitates code navigation but also simplifies the use of classes and methods within those classes. This makes it easier for the users to define new requirements for the program,

and similarly, it simplifies the implementation of these requirements into the program. Therefore, it can be argued that NFR7 is partially obtained.

10.5 User experience

System testing is used in the validation phase of the V-Model. This is where the system functionality is tested by the user. Afterward, an acceptance test is conducted by engaging the user with relevant questions. The acceptance test evaluates the user experience by testing the software product in a user environment.

The purpose of testing the product with user involvement is not only to assess whether the user requirements are fulfilled. The test also determines if the program is intuitive, easy to comprehend, and incorporates all the necessary functions that the user is seeking.

10.5.1 System test

Procedure:

1. Transfer the program via USB.
2. A shortcut to the program is created on the desktop.
3. The user is instructed to try the program on their own, and questions will be addressed after the test. The purpose of this is to evaluate user-friendliness.
4. The user and user experience are observed, and actions and the process are documented.
5. After the user completes the process, the user can ask questions.
6. The group's specific pre-prepared questions are then posed.
7. Following the questions, the questions and the test are now completed.

Observation:

In the following section, the observation of the user trying the program for the first time is described.

After the program has been transferred to the user via a USB drive and a shortcut to the program has been created on the desktop, the test begins. The user opens the program, and the initial window appears. The user presses the 'Import' button, triggering the file explorer to appear. The user locates the datasets, selects one, and clicks 'Open' in the file explorer, after which the file explorer closes. The user clicks the 'Import data' button again, as there is confusion about how many datasets can be uploaded. After a brief explanation that only one dataset can be uploaded at a time, the file explorer is closed, and the user clicks 'Next'.

A new window appears, displaying graphs and tables. The user states that it's understandable and explains what can be seen. The static and dynamic phases are described by the user in relation to the

graph, but doubts arise about the accuracy of the calculations. This is because the selected dataset contained motion artifacts. The user tries a new dataset with less motion artifacts. The program calculates and displays the correct calculations.

10.5.2 Acceptance test

In the following section, questions and the user's responses are presented.

1. *Does the program meet your expectations and fulfill your requirements?*

The user states that the requirements set for the program have been obtained and fulfilled, making the user happy and satisfied. The requirement to upload multiple datasets at once was not a direct request from the user but more of a 'nice to have.' The program functions optimally as it's now, obtaining the user's needs. The user only desires the export option if multiple datasets can be uploaded at once. When uploading only one dataset, it would be easier to enter the results manually than to export one dataset at a time. Additionally, it's a program the user envisions using in the future.

2. *How do you rate the user-friendliness of the program?*

The user states that the program is very nice and easy to use. Improvements can be made, but it would be more convenient and not necessary for the user. The user also thinks the program could be applied if it was provided without instructions.

3. *What is your opinion on the design and layout?*

The user states that the design and layout are clear in the way they are arranged. The program is simplified in the sense that it only includes the necessary functions without distracting elements. Additionally, the user finds the program very intuitive. Furthermore, the user finds it convenient that multiple windows with different datasets can be opened side by side for comparison.

4. *What do you think about the number of clicks in the program?*

The user states that the number of steps and clicks through the program is only three, which, in her opinion, is not too many. Therefore, the number of clicks works well for the user.

5. *What are your thoughts on potential improvements that could be made to the program?*

The user states that improvements to the program could include the ability to click on the graph and thereby mark two points where desired calculations can be performed. This would be useful, especially if a dataset has motion artifacts, and the program has difficulty detecting intended phases. By manually inserting phases, the dataset can be utilized effectively. Furthermore, an additional feature could be the ability to upload multiple datasets at the same time and subsequently export them. However, the user considers this function less important than the manual phase division.

Finally, the user suggests incorporating a button to visualize the calculated phases on the graph. This should serve as an option, allowing the user to decide whether the phase division should be visible or not by clicking a button.

10.6 Results

Based on the test of the functional and non-functional requirements, the summary of the requirements is illustrated in table 10.1 and 10.2 where "✓" means that the requirement is obtained. If the system has not obtained the requirement it's marked with a "X".

ID	Relation to user requirement-ID	Functional requirements	MoSCoW	Obtained
FR1	UR1	The program must transform the quaternions to Euler angles.	Mo	✓
FR2	UR2	The program must calculate, from the IMU datasets, the mean angular positions when the movements are static for the Supine Roll maneuver.	Mo	✓
FR3	UR2	The program must calculate, from the IMU datasets, the mean angular positions when the movements are static for the Dix-Hallpike maneuver.	Mo	✓
FR4	UR4	The program must calculate the mean angular velocity from the angular position during position shifts when the movements are dynamic for the Supine roll maneuver.	Mo	✓
FR5	UR4	The program must calculate the mean angular velocity by the angular position during the position shifts, when the movements are dynamic for the Dix-Hallpike maneuver.	Mo	✓
FR6	UR3	The program must calculate the time of the position shifts when the movements are dynamic for the Dix-Hallpike maneuver.	Mo	✓
FR7	UR3	The program must calculate the time of the position shifts when the movements are dynamic for the Supine Roll maneuver.	Mo	✓
FR8	UR2, UR3, UR4	The program must visualize the <i>values*</i> from UR2, UR3 and UR4 for the chosen maneuvers.	Mo	✓
FR9	UR6	The program must include graphs of the head angular position as a function of time.	S	✓
FR10	UR7	The program must be able to convert chosen <i>values*</i> from UR2, UR3 and UR4 <i>values*</i> .	Co	X

Table 10.1: The functional requirements that demonstrate the results of testing.

ID	Non-functional requirements	MoSCoW	Obtained
NFR1	The system must address the imported data in accordance with the GDPR rules.	Mo	✓
NFR2	The system must be compatible so it can communicate with file explorer.	Mo	✓
NFR3	The system must be compatible with a computer with Java installed.	Mo	✓
NFR4	The system must be compatible so it can communicate with CSV-files.	Mo	✓
NFR5	The system must have a maximum step of 5 clicks to access the desired data.	S	✓
NFR6	The system must calculate and display the desired <i>values</i> * within 10 seconds.	S	✓
NFR7	The system must be flexible so future functions can be added.	Co	✓/X

Table 10.2: The non-functional requirements that demonstrate the results of testing.

10.7 Test summary

Based on the observation of the user's initial attempt to use the program and the subsequent questions in section 10.5, it can be concluded that the program has good user-friendliness, and is simple, clear, and intuitive to use. As seen in the table 10.1 almost all requirements are obtained, which leads to a functional program for the intended user. As observed in table 10.1, the requirement FR10 isn't obtained because the requirement FR10 hasn't been implemented in the program, and therefore it hasn't been tested

Part VI

Discussion and Conclusion

11. Discussion

In the discussion, the focus will be on the addressed problem with an emphasis on problem solving and the existing solutions. Further consideration will be given to the error source and limitations. Finally, the cooperation with the external partner and the user test will be discussed.

The addressed problem

When performing the tests manually, the reliability depends on how precise and fast the head positions are executed. However, this implies that doctors and clinicians may adopt varying approaches to the procedure. No existent information system can provide the user with evidence of which the *values** have been achieved during the diagnosis. Therefore the user needs an information system that can be employed as evidence to visualize the achieved *values**.

Problem solving

An information system has been developed to provide the user with the missing evidence on which *values** were attained during the tests. The information system succeeded in calculating and printing out the achieved *values** for the chosen dataset. Furthermore, the information system successfully generated a visual representation of the position in degrees through a graph. Additionally, it is assessed that the design of the information system is user-friendly based on the user acceptance test.

However, the information system failed to load multiple files. This prolongs the time required to access all information for the user as every dataset has to be uploaded repeatedly, resulting in more clicks. Loading only one file at a time requires the user, to have a more systematic approach to document which datasets have been analyzed and identify those that are missing. In addition, a considerable investment of working hours is required to gain a broader overview of the datasets. Conceivably, it may engender misinterpretation regarding the specific data that is analyzed.

Existing solutions

There are no existing information systems on the market. Therefore the information system can not be compared to other existing programs. The absence of pre-existing systems presents the advantage of opening the possibility to create a new system and stimulating a creative process, free from the influence of pre-existing systems. The absence of a pre-existing system results in the development process of the system taking a longer duration because it was developed from scratch.

Error sources

In connection to the uploaded datasets, there may be sources of errors during the recording of the dataset. An error source can occur in the recording of data if the IMU sensor is implemented incorrectly. If the IMU sensor isn't placed centered on the head or is the IMU moved by mistake during the execution, a deviation in the expected result can occur. The deviation in the results has the potential to alter the threshold, consequently influencing the outcomes of the phases or determining whether any phase shift will be detected at all. During the diagnosis, the IMU sensor can also be affected by motion artifacts which can result in an inaccurate signal. These error sources can affect the results in the calculation of position and velocity, as an outlier may occur, which can impact the average calculation. To reduce uncertainties, the IMU sensor can be placed on the patient, however, this is not considered the most optimal option as it is perceived to have a negative impact on the patient in terms of discomfort. Therefore, placing the IMU sensor on the goggles is seen as the best and most stable solution.

Limitations

One of the limitations imposed on the program is the filename of the uploaded data. The data filename must contain either DH or SRT, which can impose restrictions on further use. If the program is handed to another user who does not use filenames containing DH or SRT, the program is not flexible in this regard. The program currently functions optimally with these specific filenames, as it aligns with what the external partner has named the files. Due to the project's timeframe, it has not been possible to implement the option to upload files with different names. To meet the requirements of NFR7 as seen in table 10.2, the program should be able to upload files with alternative names in the future.

It can be argued that requirement NFR7 is only partially obtained at the current time. This suggests that NFR7 may have been too imprecise in formulating the requirement, making it challenging to test. The requirement could have been tested by providing it to other programmers to assess its comprehensibility and to evaluate the feasibility of incorporating their contributions into the program.

Cooperation with external partner

In collaboration with the external partner, establishing a mutual understanding of the product requirements is crucial. Achieving mutual understanding can be a challenging process due to potential miscommunication between the user and the developer, often resulting from diverse educational backgrounds. The challenge was evident during the development of requirements, where identifying and prioritizing specific criteria proved difficult. Consequently, this led to an extended development process characterized by multiple iterations of the requirements.

For instance, a discussion about incorporating velocity as a function of time into the information system. After consideration, it was removed as deriving velocity from numerical data in the table was deemed as a better solution than presenting it visually in a graph. The advantage of investing extra time to form a shared understanding refines the product, resulting in the desired program. The shared knowledge between a developer and a user with a diverse educational background offers the advantage of providing broader insights for each part

User test

As mentioned in section 10.5, no instructions were provided when the user tested the program. There is a debate on whether the test should have been conducted with guidelines. This approach could have resulted in the user having a better understanding from the outset, avoiding minor missteps during the test. The way the test was conducted revealed deviations in the usability of the program that might not have been discovered if the user had been guided through each step of the process.

If the project timeframe had been longer, it would have been advantageous to test with more users than the intended user. This would have provided the advantage of examining the usability across a broader spectrum, as multiple individuals could evaluate the information system. This approach could facilitate discussions about which features could be added or omitted. The disadvantages of testing with multiple users can lead to new requirements that are not crucial for the functionality of the program and the intended user. This also requires the developer to engage in a new process to determine which requirements are beneficial and which ones are unnecessary

11.1 Future perspective

In future development of the program, several aspects could be incorporated into the program. In the case of the program struggling to detect expected phases, a solution could be that the user manually has the option to insert phases if a dataset contains a significant amount of motion artifacts. This could be implemented through a button; when pressed, it would display the calculated phases and allow the user to navigate through them by moving the phases. It's worth noting that this functionality was not a requirement but is a suggestion contributed by our external partner after the finalization of the program. Throughout the development, the group primarily worked with datasets where there were fewer motion artifacts than there were in some of the other datasets. Therefore, this aspect was not considered in the initial process but could be valuable for further development.

Another improvement for the further development of the program could be to enable it to incorporate multiple datasets simultaneously. This would allow users to analyze several datasets at once, eliminating the need to load them one by one, as the current program requires. This would save users time, as multiple datasets could be uploaded simultaneously. This would also align with the low-fidelity prototype developed at the beginning of the process. In the low-fidelity prototype, different maneuvers of the patient could be selected, and the user could then visualize and analyze how the maneuvers were executed.

In the low-fidelity prototype, there is a function to convert the dataset into a CSV file. This function will also be highly useful in the future. It's essential to note that the external partner is not interested in converting one dataset at a time, so the option to import multiple datasets at once should be available.

By using a database, it allows multiple users to handle the data without conflicts. Additionally, databases support data security and ensure that the data remains concrete and concise. A database can enable easy and quick searching of the data. This is effective if the user has a substantial amount of information. Databases reduce the need for repeated data storage, resulting in minimized storage space. Additionally, the ability to store the calculated values will reduce the need for recalculating every time the same dataset is uploaded. This ensures that the program only performs calculations once for each unique dataset. While databases have the potential to positively impact a product, it's not employed in this project, because other functions have been prioritized.

12. Conclusion

Based on the knowledge acquired throughout the project and the results of the conducted tests, it can be concluded, that the program can calculate the desired values, mean angular position, mean angular velocity, and duration for Dix-Hallpike maneuver, and Supine Roll maneuver. The calculated values can be used to evaluate whether the maneuvers for the diagnosis of BPPV have been performed precisely.

The program contains the most important functionalities. As a result, some of the functionalities were not implemented, however, this did not adversely impact the final solution to the problem. In the test of functional requirements, 9 out of 10 were obtained and in the test of non-functional 9 out of 10 were obtained after analyzing the results. The results form the basis for the information system to obtain the significant requirements.

Overall, a digital information system has been designed and implemented which solves the problem statement.

How can a digital information system be developed to calculate values which allow the user to analyze whether the maneuvers for the diagnosis of BPPV, are performed precisely?

Bibliography

- [1] Auris Nasus Larynx. “Benign paroxysmal positional vertigo”. In: (2022). DOI: 10.1016/j.anl.2022.03.012. URL: <https://pubmed.ncbi.nlm.nih.gov/35387740/> (visited on 12/15/2023).
- [2] Kim JS Kim HJ Park J. “Update on benign paroxysmal positional vertigo”. In: (2021). DOI: 10.1007/s00415-020-10314-7.
- [3] Mikkel Holmelund. *Ørestenssvimmelhed (BPPV)*. URL: https://oerelaegerne.dk/wp-content/uploads/2014/09/11494-7_final.pdf (visited on 12/13/2023).
- [4] Lægehåndbogen. *Benign Paroxystisk Positionel Vertigo (BPPV)*. 2022. URL: <https://www.sundhed.dk/sundhedsfaglig/laegehaandbogen/neurologi/tilstande-og-sygdomme/oevrige-sygdomme/benign-positionel-vertigo/> (visited on 12/13/2023).
- [5] Mikkel Holmelund and Jette Koldning Kristensen. *Benign positionel vertigo (BPPV) - Patienthåndbogen på sundhed.dk*. 2022. URL: <https://www.sundhed.dk/borger/patienthaandbogen/hjerne-og-nerver/sygdomme/oevrige-sygdomme/benign-positionel-vertigo-bppv/> (visited on 12/15/2023).
- [6] Shayna R Cole and Julie A Honaker. “Benign paroxysmal positional vertigo: Effective diagnosis and treatment.” In: *Cleveland Clinic Journal of Medicine* 89.11 (2022), pp. 653–662. DOI: 10.3949/ccjm.89a.21057.
- [7] Mayo Clinic Staff. *Dizziness*. 2022. URL: <https://www.mayoclinic.org/diseases-conditions/dizziness/symptoms-causes/syc-20371787> (visited on 09/25/2023).
- [8] Seong Hae Jeong. “Benign Paroxysmal Positional Vertigo Risk Factors Unique to Perimenopausal Women”. In: *Frontiers in Neurology* 11 (Oct. 2020), p. 589605. ISSN: 16642295. DOI: 10.3389/FNEUR.2020.589605.
- [9] Kristen Davies. *The Inner Ear*. 2022. URL: <https://teachmeanatomy.info/head/organs/ear/inner-ear/> (visited on 09/12/2023).
- [10] Elizabeth Crawford. *What are the otoliths?* URL: <https://www.vestibularhealth.ca/blog/what-are-the-otoliths> (visited on 09/12/2023).
- [11] Briony Adams. *The Vestibulocochlear Nerve (CN VIII)*. 2023. URL: <https://teachmeanatomy.info/head/cranial-nerves/vestibulocochlear/> (visited on 09/12/2023).

- [12] Lorne S. Parnes, Sumit K. Agrawal, and Jason Atlas. “Diagnosis and management of benign paroxysmal positional vertigo (BPPV)”. In: *CMAJ: Canadian Medical Association Journal* 169 (7 Sept. 2003), p. 681. ISSN: 08203946. URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC202288/> (visited on 12/15/2023).
- [13] Britta Smith. *Posterior Canal Benign Paroxysmal Positional Vertigo (BPPV)*. September 2019. 2019. URL: https://www.neuropt.org/docs/default-source/vsig-physician-fact-sheets/posterior-canal-benign-paroxysmal-positional-vertigo.pdf?sfvrsn=6bda5343_0 (visited on 12/15/2023).
- [14] L Califano et al. “Anterior canal BPPV and apogeotropic posterior canal BPPV: two rare forms of vertical canalolithiasis”. In: *ACTA otorhinolaryngologica italica* 34.3 (2014), p. 189. URL: <https://pubmed.ncbi.nlm.nih.gov/24882928/> (visited on 12/15/2023).
- [15] Anita Bhandari et al. “Modified interpretations of the supine roll test in horizontal canal BPPV based on simulations: How the initial position of the debris in the canal and the sequence of testing affects the direction of the nystagmus and the diagnosis”. In: *Frontiers in Neurology* 13 (2022), p. 881156. DOI: 10.3389/fneur.2022.881156.
- [16] Lorne S Parnes, Sumit K Agrawal, and Jason Atlas. “Diagnosis and management of benign paroxysmal positional vertigo (BPPV)”. In: *Cmaj* 169.7 (2003), pp. 681–693. URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC202288/> (visited on 12/15/2023).
- [17] Simon Harris. *Inertial Measurement Unit (IMU) – An Introduction*. 2023. URL: <https://www.advancednavigation.com/tech-articles/inertial-measurement-unit-imu-an-introduction/> (visited on 09/19/2023).
- [18] SBG Systems. *What is an Inertial Measurement Unit - IMU?* URL: <https://www.sbg-systems.com/inertial-measurement-unit-imu-sensor/> (visited on 09/19/2023).
- [19] Ben Kenwright. “A beginners guide to dual-quaternions: what they are, how they work, and how to use them for 3D character hierarchies”. In: (2012). URL: http://wscg.zcu.cz/WSCG2012/!_WSCG2012-Communications-1.pdf#page=15 (visited on 12/15/2023).
- [20] Graphics Fandom. *Conversion between quaternions and Euler angles*. URL: https://graphics.fandom.com/wiki/Conversion_between_quaternions_and_Euler_angles (visited on 12/12/2023).
- [21] Zhihao Shen, Armagan Elibol, and Nak Young Chong. “Understanding nonverbal communication cues of human personality traits in human-robot interaction”. In: *IEEE/CAA Journal of Automatica Sinica* 7.6 (2020), pp. 1465–1477. DOI: 10.1109/JAS.2020.1003201.
- [22] Michael Von Brevern et al. “Benign paroxysmal positional vertigo: diagnostic criteria”. In: *Journal of Vestibular Research* 25.3-4 (2015), pp. 105–117. DOI: 10.3233/VES-150553.

- [23] Liz Fuemmeler. *More than a Frenzel: Do more for your patients with Video Frenzel goggles*. 2023. URL: <https://www.interacoustics.com/blog/video-frenzel-goggles> (visited on 09/19/2023).
- [24] Elizabeth Crawford. *Why we use infrared goggles to assess and treat vertigo*. URL: <https://www.vestibularhealth.ca/blog/infrared-goggles-for-vertigo> (visited on 09/19/2023).
- [25] Mads Klokke and Interacoustics. *TRV Chair, BPPV treatment made efficient*. 2023. URL: https://www.interacoustics.com/images/files/8523046-US_trv_chair_brochure_v1_07_2020.pdf (visited on 09/19/2023).
- [26] Interacoustics. *TRV Chair, BPPV treatment made efficient*. 2023. URL: <https://www.interacoustics.com/balance-testing-equipment/trv-chair> (visited on 09/19/2023).
- [27] Morten Falshøj Pedersen and et al. "Treatment of benign paroxysmal positional vertigo with the TRV reposition chair". In: *The Journal of International Advanced Otolaryngology* 16.2 (2020), p. 176. DOI: 10.5152/iao.2020.6320.
- [28] Hengyong Tang and Wei Li. "Advances in the diagnosis and treatment of benign paroxysmal positional vertigo." In: (2017). URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5609213/> (visited on 09/19/2023).
- [29] JavaTpoint. *V-Model*. 2021. URL: <https://www.javatpoint.com/software-engineering-v-model> (visited on 12/05/2023).
- [30] Toturials Point. *SDLC - V-Model*. URL: https://www.tutorialspoint.com/sdlc/sdlc_v_model.htm (visited on 12/05/2023).
- [31] Pi-Sheng Deng and Cynthia L Fuhr. "Using an object-oriented approach to the development of a relational database application system". In: *Information & management* 29.2 (1995), pp. 107–121. URL: <https://www.sciencedirect.com/science/article/pii/037872069500020W> (visited on 12/15/2023).
- [32] IBM. *Rational Software Architect 9.7.0*. september 2023. 2023. URL: <https://www.ibm.com/docs/en/rational-soft-arch/9.7.0> (visited on 10/25/2023).
- [33] usability. *Prototyping*. October 25, 2023. 2023. URL: <https://www.usability.gov/how-to-and-tools/methods/prototyping.html> (visited on 10/25/2023).
- [34] Avroha Shuhalii. *What is the difference between low and high fidelity prototypes? Types of prototypes. Who is responsible for creating them?* 2020. URL: <https://bootcamp.uxdesign.cc/what-is-the-difference-between-low-and-high-fidelity-prototypes-b1f3612f85f7> (visited on 11/25/2023).

- [35] Technical Writer Alexander S. Gillis and Editor. *object-oriented programming (OOP)*. 2021. URL: https://www.techtarget.com/searchapparchitecture/definition/object-oriented-programming-OOP?fbclid=IwAR1yoGeuny_QK98we8XYjsiHp2M4raj3d3-XRTQC7VIaNA2TA48EGhYhtjk (visited on 11/09/2023).
- [36] tutorialspoint. *Design Pattern — Overview*. URL: https://www.tutorialspoint.com/design_pattern/design_pattern_overview.htm (visited on 12/05/2023).
- [37] Robert Sheldon. “Model-view-controller (MVC)”. In: *TechTarget* (2023). URL: <https://www.techtarget.com/whatis/definition/model-view-controller-MVC> (visited on 12/15/2023).
- [38] geeksforgeeks. *Singleton Design Pattern — Implementation*. 2023. URL: <https://www.geeksforgeeks.org/singleton-design-pattern/> (visited on 12/05/2023).
- [39] Kate Brush. “MoSCoW method”. In: *TechTarget, April* (2020). URL: <https://www.techtarget.com/searchsoftwarequality/definition/MoSCoW-method> (visited on 12/15/2023).
- [40] Geertjan Wielenga. *Beginning netbeans ide: For java developers*. Apress, 2015.
- [41] James Gosling, David Colin Holmes, and Ken Arnold. *The Java programming language*. 2005. URL: <https://www.acs.ase.ro/Media/Default/documents/java/ClaudiuVinte/books/ArnoldGoslingHolmes06.pdf>.
- [42] Gregory G Slabaugh. “Computing Euler angles from a rotation matrix”. In: *Retrieved on August 6.2000* (1999), pp. 39–63. URL: <https://eecs.qmul.ac.uk/~gslabaugh/publications/euler.pdf> (visited on 12/12/2023).
- [43] Datatilsynet. *Hvad er personoplysninger?* URL: <https://www.datatilsynet.dk/hvad-siger-reglerne/grundlaeggende-begreber/hvad-er-personoplysninger> (visited on 10/18/2023).
- [44] Ahmed H Yousef, Tamer A El-lateef, and Mona F Ismail. “Benchmarking and performance enhancement framework for multi-staging object-oriented languages”. In: *Ain Shams Engineering Journal* 4.2 (2013), pp. 241–257. URL: <https://www.sciencedirect.com/science/article/pii/S209044791200072X> (visited on 12/05/2023).

Part VII

Appendix

A. Particle repositioning maneuvers

In this appendix, the procedure for the three maneuvers for treatment, Epley, Semont, and log roll will be explained. Throughout the maneuver, the patient's head and body must be kept in the correct position. While the procedure is performed, it is important to observe the pattern of nystagmus to ensure that the otoliths aren't moved to a different semicircular canal.

To perform the Epley maneuver

If the otoliths are located in the vertical semicircular canals (in the posterior or anterior semicircular canals), the Epley maneuver is used. The otoliths are moved through the vertical semicircular canals and back to the utricle. The Epley maneuver is performed as described:

1. At first, the patient is placed in a seated position, and their head is turned 45° to the side where the otoliths are suspected to be.
2. Secondly, the patient is then moved to a supine position, and the head is extended downward.
3. The patient's head is then moved to the other side, still extended downward, and kept in a 45° position. The patient must be kept in the correct plan throughout the maneuver.
4. The patient's chin is then placed at 45° toward the unaffected ear.
5. At last, the patient is placed in a seated position with their head turned over their shoulder.

To perform the Semont maneuver

An alternative maneuver for treating the otoliths located in the vertical semicircular canals (in the posterior or anterior semicircular canals) is called the Semont maneuver. It is performed as described:

1. Firstly, the patient is in a seated position and the head is at 45° turned away from the ear with suspected otoliths in the posterior or anterior canals.
 2. Secondly, the patient is then moved quickly to the side with the nose facing the ceiling
 3. Lastly, still at 45° the patient is moved quickly to the other side with the nose facing the ground.
- After the patient is placed in a seated position still with the head at 45° away from the ear with suspected otoliths in the posterior or anterior canals.

To perform the log roll maneuver

For treating otoliths suspected in the horizontal canal, the log roll or the Lempert maneuver can be used. The log roll maneuver is performed as described:

1. First, the patient is placed in a supine position with their head turned 90° toward the ear with the suspected otoliths in the horizontal semicircular canal.
2. Secondly, the patient's head is turned back to the center, elevated 30° .
3. The patient is then moved 90° to the side with the unaffected ear.
4. Then the patient is moved onto the elbows while they are flexed.
5. Last, the patient is placed back onto their back toward the ear suspected of horizontal otoliths, which also completes a 360° rotation.

B. Visualizing of quaternions

During the project, datasets were made available by the external partner. The purpose was to analyze and recalculate the data to gain a deeper insight and a better understanding of the data. The graph in figure B.1 represents the quaternion at the y-axis while the x-axis represents the number of samples.

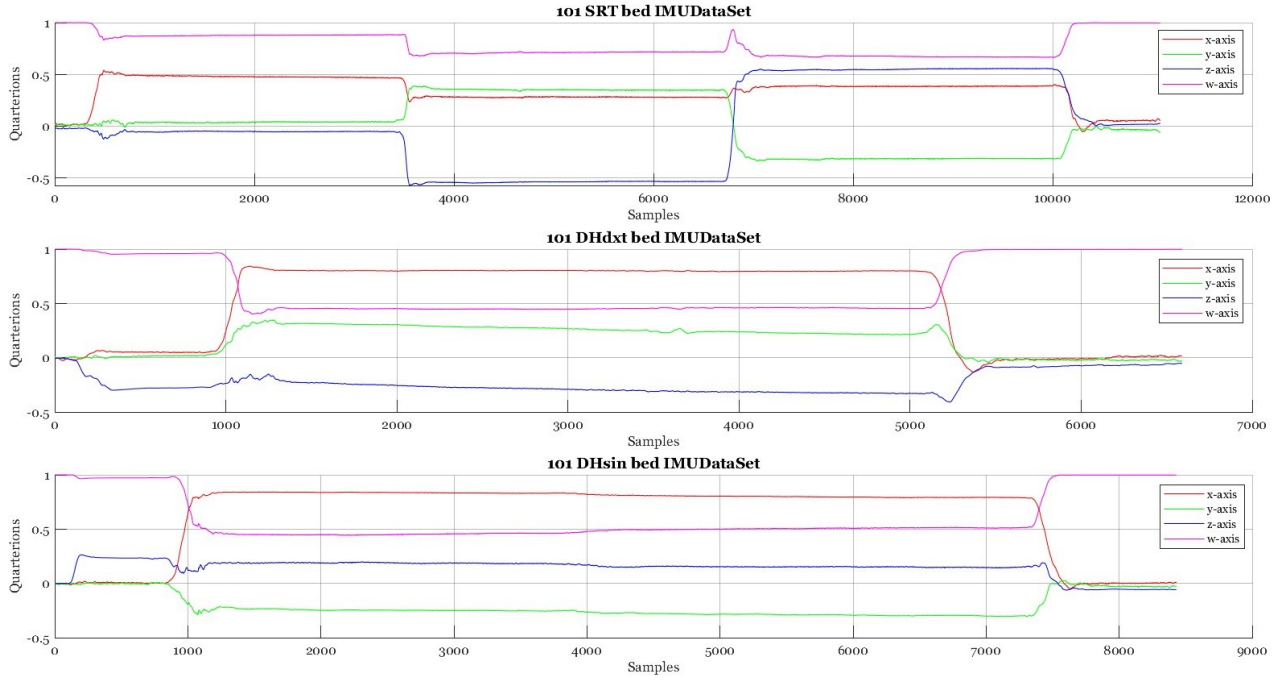


Figure B.1: Visualization of raw data from a Supine Roll maneuver, Dix-Hallpike right, and Dix-Hallpike left. The figure is made in MATLAB.

C. Visualization of precise head angular position

In figure C.1 the precise head angular position over time is visualized. The three graphs represent the precise head angular position in the Supine Roll maneuver, Dix-Hallpike right and left. The angles are indicated as either positive or negative values indicating whether the patient is turning their head clockwise to the right or counterclockwise to the left. The duration of diagnostic maneuvers varies based on the specific maneuvers performed.



Figure C.1: Illustration of the theoretical head angular position over time for the different maneuvers. This is created based on the values in table 3.5.

D. Prototypes

To develop a prototype for the external partner, ongoing meetings were held to gain an understanding of the product's requirements and specifications.

First prototype:

The approach for the first prototype was drawn on a blackboard. The system starts with a login. The user can import all data, and add new data for all maneuvers. The user will search for a patient, and a pop-out menu will show. The menu contains the different maneuvers that show the values and graph. The user now has the possibility to convert to Excel for further use and start over with a new patient. Which is made based on information from our collaborator. The first prototype is visualized in figure D.1.

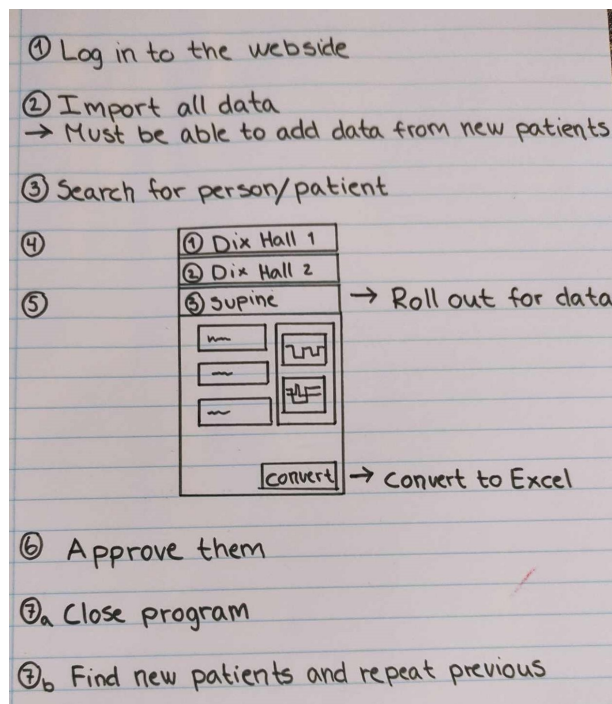


Figure D.1: Illustration of the first prototype.

Second prototype:

In the second prototype, further development has been made on how the user should be able to search for the desired data. Additionally, details are provided on how the maneuvers and corresponding IDs are aligned in Excel to collect the values from different IDs and maneuvers in the same file during the 'Convert all data to Excel file' function. The second prototype is visualized in figure D.2.

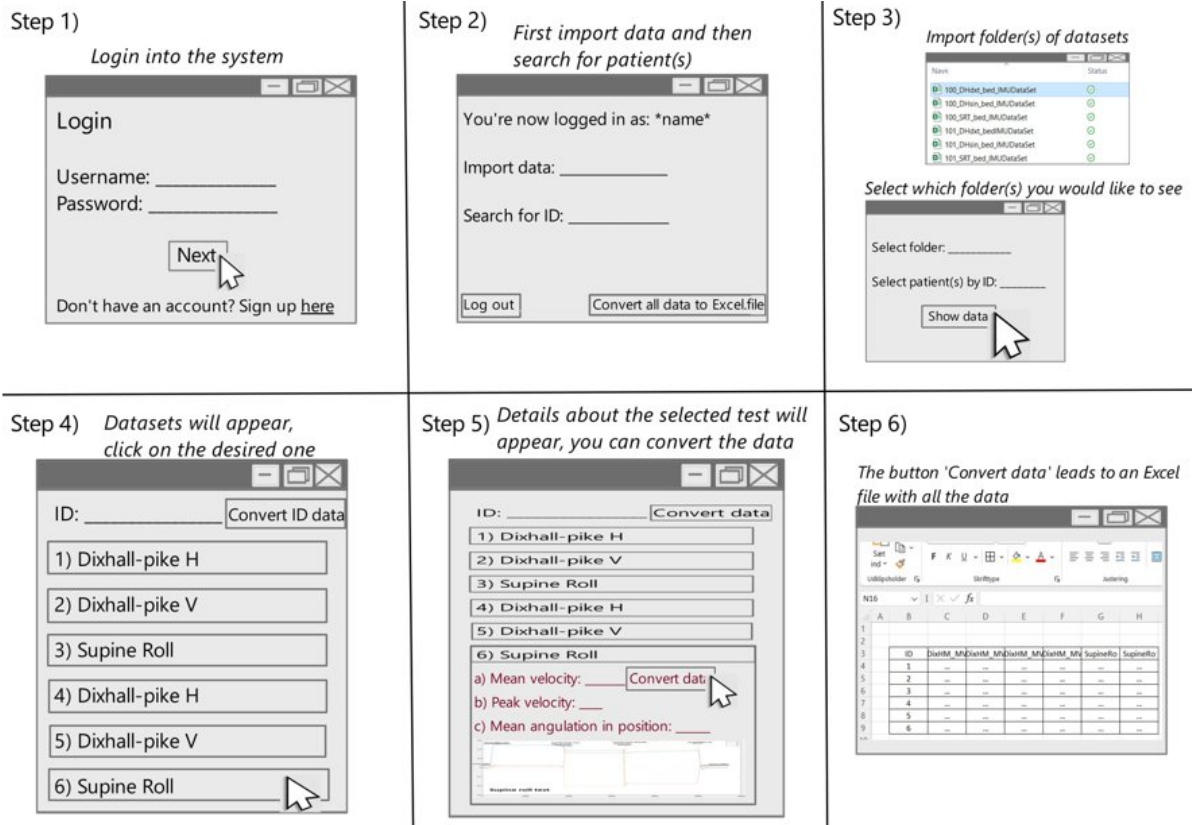


Figure D.2: Illustration of the second prototype.

Third prototype:

The third prototype is based on the second prototype but with adjustments and more functionalities. In this prototype, one of the new functionalities is that the user can choose all IDs marked in a checkbox. Furthermore, the user has the flexibility to choose the specific maneuver for which they wish to visualize values. It's also marked in checkboxes. The third prototype is visualized in figure D.3.

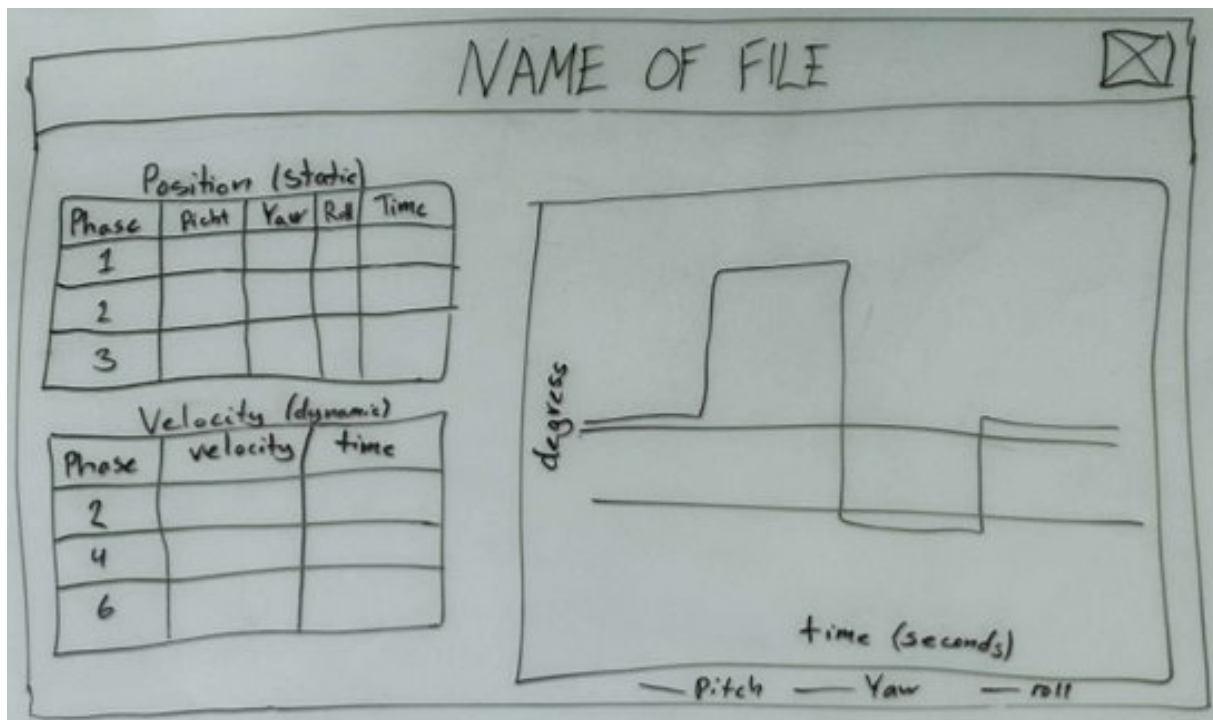


Figure D.4: Illustration of 'Window2' in the final prototype.